

# Optimizing Online Business Security with Blockchain Technology

Graduation Project II Report – Submitted in partial fulfillment of the  
requirements for the Bachelor's Degree in Faculty of Artificial  
Intelligence Engineering – Intelligent Information Security Systems  
Engineering

## Prepared by

Hamza Al-Okla

Saleh Al-Halaqi

Majdi Al-Masri

## Supervised by

Dr. Eng. Wassim Juneidi

Eng. Ahmad Shekha

June 2026

# تحسين أمان الأعمال التجارية عبر الإنترنت باستخدام تقنية سلسلة الكتل

قدّم استكمالاً لمتطلبات الحصول على درجة 2تقرير لمشروع التخرج  
البكالوريوس في هندسة الذكاء الاصطناعي - هندسة نظم أمن المعلومات الذكية

## إعداد

حمزه عمر العقلة  
صالح أحمد الحلقي  
مجدي عواد المصري

## إشراف

د. وسيم جنيدي  
م. أحمد شخه

حزيران 2026

# SUPERVISION CERTIFICATION

I Certify that the preparation of this project entitled

[Optimizing Online Business Security With Blockchain Technology]

Prepared by

[Hamza Al-Okla – Saleh Al-Halaqi – Majdi Al-Masri]

was made under my supervision at Faculty of Artificial Intelligence Engineering in partial  
Fulfillment of the Requirements for the Degree of Bachelor of Intelligent Information Security  
Systems Engineering

Name: .....

Signature: .....

Date: .....

# مَشْكُورٌ وَتَقْدِيرٌ

نتقدم بخالص الشكر وعظيم الامتنان لكل من ساهم في إنجاز هذا العمل ودعمه علمياً وأكاديمياً.

نتوجه بجزيل الشكر والتقدير إلى الدكتور وسيم جندي، المشرف على هذا المشروع، لما قدمه من إشراف علمي متميز، وتوجيهات أكاديمية قيّمة، ودعم مستمر طوال مراحل تنفيذ المشروع. لقد كان لخبرته، وحرصه الدائم على تحفيزنا وتشجيعنا، الأثر الكبير في تعزيز ثقتنا بأنفسنا، ودفعنا نحو تقديم أفضل ما لدينا، وإخراج هذا المشروع بالصورة العلمية اللائقة.

كما نتقدم بخالص الشكر إلى الدكتورة كريستين زينية، على ما قدمته من ملاحظات علمية ببناء، ونصائح أكاديمية أسهمت في تحسين جودة المشروع وتعزيز مضمونه العلمي.

ونتوجه أيضاً بالشكر والتقدير إلى المهندس أحمد شيخه، لما قدموا من دعم فني وإرشادات عملية ساعدتنا في تجاوز العديد من التحديات التقنية خلال تنفيذ المشروع.

ولا يفوتنا أن نعبر عن امتناننا لجميع أعضاء الهيئة التدريسية في الجامعة السورية الخاصة، الذين كان لهم دور أساسي في بناء أساسنا العلمي والمعرفي، وكذلك لكل من قدّم لنا الدعم والمساندة خلال مسيرتنا الأكاديمية.

وأخيراً، نشكر كل من وقف إلى جانبنا وشجعنا على الاستمرار والعطاء، سائلين الله التوفيق للجميع.

هَذَا

إِهْلَاء

إِهْلَاء

# Abstract

This project presents the design and implementation of a blockchain-based e-commerce security platform developed to improve transaction integrity, distributed trust, and secure communication within modern web applications. Traditional centralized e-commerce systems suffer from several security limitations such as data tampering, single points of failure, and limited transaction traceability. To address these challenges, the proposed system integrates blockchain technology with distributed system architecture in order to provide secure transaction validation and decentralized synchronization.

The proposed platform was implemented using ASP.NET Core, Entity Framework Core, SQL Server LocalDB, RESTful APIs, and distributed blockchain services. The system follows the MVC architectural pattern and operates within a simulated distributed blockchain environment composed of multiple interconnected nodes communicating through peer-to-peer synchronization mechanisms.

The implemented solution supports blockchain validation, cryptographic hash generation, distributed block replication, audit logging, and real-time synchronization between blockchain nodes. The platform additionally provides a blockchain explorer interface for monitoring blockchain records, transaction history, and distributed synchronization status.

The evaluation results demonstrated that the proposed system successfully achieved secure transaction processing, blockchain consistency, distributed synchronization, and decentralized validation across multiple nodes. The project also showed that integrating blockchain technology with modern web architectures can significantly improve data integrity, tamper resistance, and distributed trust in e-commerce environments.

The proposed platform represents a practical implementation of blockchain concepts within a distributed web application environment and provides a scalable foundation for future blockchain-based transaction systems.

# Table of Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| <b>Abstract</b> .....                                              | 8  |
| <b>Chapter 1: Introduction</b> .....                               | 13 |
| <b>1-1.Introduction</b> .....                                      | 14 |
| 1-2.The Research Problem .....                                     | 14 |
| <b>1-3.Research Objective</b> .....                                | 15 |
| <b>1-4.Technical Challenges</b> .....                              | 15 |
| 1-4.1.Network Latency & Consensus Overhead.....                    | 15 |
| 1-4.2.Digital Key Management & Security .....                      | 15 |
| 1-4.3.Chain Forks & Data Synchronization.....                      | 16 |
| 1-4.4.Sybil Attack & Weight Manipulation.....                      | 16 |
| <b>1-5.Literature Review</b> .....                                 | 16 |
| <b>1-6.State-of-the-art (SOTA)</b> .....                           | 18 |
| 1-6.1.Distributed Zero Trust... ..                                 | 18 |
| 1-6.2.Proof of Reputation (PoR).....                               | 18 |
| 1-6.3.Edge Computing.....                                          | 18 |
| 1-6.4.Sybil Resistance.....                                        | 18 |
| <b>1-7. Conclusion</b> .....                                       | 18 |
| <b>Chapter 2: Theoretical Framework</b> .....                      | 20 |
| <b>2-1.Introduction</b> .....                                      | 21 |
| <b>2-2.Blockchain Technology</b> .....                             | 21 |
| 2-2.1.The Fundamental Concept of Blockchain.....                   | 21 |
| 2-2.2.Consensus Mechanisms.. ..                                    | 23 |
| 2-2.3.Smart Contracts.....                                         | 24 |
| 2-2.4.Security Breaches and Attacks on Blockchain Technology.....  | 25 |
| 2-2.5.Information Security Measures in Blockchain Technology ..... | 29 |
| <b>2-3.Conclusion</b> .....                                        | 32 |
| <b>Chapter 3: Setting Up the Development Environment</b> .....     | 33 |
| <b>3-1.Introduction</b> .....                                      | 34 |
| <b>3-2.Hardware Requirements</b> .....                             | 35 |
| <b>3-3.Software Requirements</b> .....                             | 37 |
| <b>3-4.Development Environment Setup</b> .....                     | 38 |
| <b>3-5.Backend Environment Configuration</b> .....                 | 40 |
| <b>3-6.Database Environment Setup</b> .....                        | 42 |

|                                                     |    |
|-----------------------------------------------------|----|
| 3-7.Blockchain Environment Configuration.....       | 44 |
| 3-8.Distributed Node Setup .....                    | 46 |
| 3-9.API and Communication Configuration.....        | 47 |
| 3-10.Security and Validation Environment .....      | 49 |
| 3-11.Frontend Environment Configuration.....        | 50 |
| 3-12.System Workflow .....                          | 52 |
| 3-13.Conclusion .....                               | 53 |
| Chapter 4: The Proposed Solution.....               | 54 |
| 4-1.Introduction.....                               | 55 |
| 4-2.Proposed System Overview.....                   | 56 |
| 4-3.Proposed System Architecture.....               | 57 |
| 4-4.MVC-Based Application Design .....              | 59 |
| 4-4.1.Model Layer.....                              | 60 |
| 4-4.2.View Layer.....                               | 60 |
| 4-4.3.Controller Layer.....                         | 61 |
| 4-5.Blockchain-Based Solution Design .....          | 62 |
| 4-6.Distributed Node Communication.....             | 63 |
| 4-7.Database Design and Storage Architecture .....  | 65 |
| 4-8.Security and Validation Mechanisms.....         | 67 |
| 4-9.REST API and Communication Protocols .....      | 69 |
| 4-10.Frontend Design and User Interaction.....      | 70 |
| 4-11.Complete System Workflow.....                  | 72 |
| 4-12.Proposed Algorithms and Validation Logic ..... | 73 |
| 4-13.Conclusion .....                               | 74 |
| Chapter 5: Comparisons and Results.....             | 75 |
| 4-1.Introduction.....                               | 76 |
| 4-2.System Execution Results.....                   | 76 |
| 4-3.Blockchain Validation Results .....             | 80 |
| 4-4.Distributed Synchronization Results.....        | 82 |
| 4-5.REST API Testing Results .....                  | 83 |
| 4-6.Security and Validation Results.....            | 85 |
| 4-7.Database Persistence Results .....              | 86 |
| 4-8.Performance and Stability Results .....         | 88 |
| 4-9.Comparison with Previous Studies.....           | 89 |
| 4-10.Discussion and Analysis.....                   | 90 |
| 4-11.Conclusion .....                               | 91 |
| Chapter 6: Conclusion .....                         | 92 |

|                                                            |            |
|------------------------------------------------------------|------------|
| <b>6-1.Introduction.....</b>                               | <b>93</b>  |
| <b>6-2.Project Summary .....</b>                           | <b>93</b>  |
| <b>6-3.Main Contributions of the Project .....</b>         | <b>94</b>  |
| 6-3.1.Distributed Blockchain Architecture .....            | 94         |
| 6-3.2.Blockchain Validation Mechanisms .....               | 94         |
| 6-3.3.Distributed Synchronization .....                    | 95         |
| 6-3.4.MVC and Blockchain Integration .....                 | 95         |
| 6-3.5.Security and Audit Logging .....                     | 95         |
| <b>6-4.Strengths of the Proposed System .....</b>          | <b>95</b>  |
| <b>6-5.Limitations of the Project.....</b>                 | <b>96</b>  |
| <b>6-6.Future Work and Recommendations .....</b>           | <b>97</b>  |
| 6-6.1.Advanced Consensus Algorithms .....                  | 97         |
| 6-6.2.Cloud-Based Deployment .....                         | 97         |
| 6-6.3.Smart Contract Integration.....                      | 97         |
| 6-6.4.Advanced Cryptographic Security .....                | 97         |
| 6-6.5.Real Blockchain Network Integration.....             | 97         |
| 6-6.6.Artificial Intelligence Integration .....            | 98         |
| <b>6-7.Personal Perspective and Research Insights.....</b> | <b>98</b>  |
| <b>6-8.Final Conclusion.....</b>                           | <b>98</b>  |
| <b>References .....</b>                                    | <b>100</b> |

## List of Tables

|                                                                                                                            |           |
|----------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Table 1 : An Analytical Comparison of Key Consensus Mechanisms.....</b>                                                 | <b>24</b> |
| <b>Table 2 : The Most Representative and Common Attacks on Blockchain .....</b>                                            | <b>26</b> |
| <b>Table 3 : Examples of Major Security Incidents During the Evolution of Blockchain .....</b>                             | <b>29</b> |
| <b>Table 4 : Defensive Measures for Each Blockchain Layer .....</b>                                                        | <b>30</b> |
| <b>Table 5 : Design and Implementation of Innovative Defensive Solutions Using Smart Contracts.....</b>                    | <b>31</b> |
| <b>Table 6 : Hardware Specifications of the Development Environment.....</b>                                               | <b>35</b> |
| <b>Table 7 : The most important software technologies used.....</b>                                                        | <b>37</b> |
| <b>Table 8 : Main System Components and Their Responsibilities .....</b>                                                   | <b>56</b> |
| <b>Table 9 : Functional Comparison of the Proposed System with Traditional and Previous Blockchain-Based Systems .....</b> | <b>89</b> |

# List of Figures

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| Figure 1 : Blockchain Technology Generations .....                                               | 21 |
| Figure 2 : The Fundamental Architecture of a Blockchain Network.....                             | 22 |
| Figure 3 : Interaction of DApps and Smart Contract for Consumer Electronics. ....                | 25 |
| Figure 4 : Hardware Environment Diagram .....                                                    | 36 |
| Figure 5 : Software Stack Diagram.....                                                           | 38 |
| Figure 6 : Development environment setup flow diagram .....                                      | 40 |
| Figure 7 : Backend architecture configuration diagram.....                                       | 42 |
| Figure 8 : Database environment architecture diagram .....                                       | 44 |
| Figure 9 : Blockchain architecture overview diagram .....                                        | 45 |
| Figure 10 : Distributed blockchain network infographic.....                                      | 47 |
| Figure 11 : REST API communication flow diagram .....                                            | 48 |
| Figure 12 : Security validation workflow diagram .....                                           | 50 |
| Figure 13 : Frontend architecture diagram overview .....                                         | 51 |
| Figure 14 : Complete blockchain system workflow diagram.....                                     | 53 |
| Figure 15 : System Overview Architecture Diagram .....                                           | 57 |
| Figure 16 : Layered System Architecture Diagram .....                                            | 59 |
| Figure 17 : MVC Security-Oriented Architecture Diagram .....                                     | 61 |
| Figure 18 : Blockchain Architecture Diagram .....                                                | 63 |
| Figure 19 : Distributed Blockchain Network Diagram .....                                         | 65 |
| Figure 20 : Database Environment Architecture Diagram .....                                      | 67 |
| Figure 21 : Security Validation Workflow Diagram .....                                           | 69 |
| Figure 22 : REST API Communication Diagram .....                                                 | 70 |
| Figure 23 : Frontend Environment Architecture Diagram.....                                       | 71 |
| Figure 24 : Complete System Workflow Diagram .....                                               | 73 |
| Figure 25 : Execution of Node-A in the Distributed Blockchain Environment .....                  | 77 |
| Figure 26 : Execution of Node-B in the Distributed Blockchain Environment .....                  | 77 |
| Figure 27 : Execution of Node-C in the Distributed Blockchain Environment .....                  | 77 |
| Figure 28 : Main Browser Interface of the Proposed Platform .....                                | 78 |
| Figure 29 : Blockchain Explorer Displaying Distributed Blockchain Records .....                  | 78 |
| Figure 30 : System Execution Result Diagram .....                                                | 80 |
| Figure 31 : Blockchain Explorer Displaying Blockchain Transactions and Distributed Records ..... | 81 |
| Figure 32 : Blockchain Validation Result Diagram .....                                           | 82 |
| Figure 33 : Distributed Synchronization Result Diagram .....                                     | 83 |
| Figure 34 : REST API Result Workflow Diagram.....                                                | 85 |
| Figure 35 : Security Validation Result Workflow Diagram .....                                    | 86 |
| Figure 36 : Database Persistence Result Diagram.....                                             | 87 |
| Figure 37 : System Performance Overview Diagram .....                                            | 89 |

# **Chapter 1: Introduction**

## 1-1. Introduction

In light of the rapid digital revolution witnessed in our contemporary world, the protection of electronic data has become one of the most critical pillars upon which the strategies of both nations and institutions are built. The field of Information Security represents the cornerstone of this digital civilization; it functions as the immune system for the digital infrastructure that manages all aspects of modern life, including the economy, healthcare, education, and government services.

Within this framework, encryption technologies emerge as one of the most vital tools in the information security arsenal, forming the primary line of defense for data—whether stored in databases or transmitted across communication networks. By utilizing advanced mathematical algorithms, encryption ensures confidentiality and integrity, thereby preventing unauthorized access or data tampering. These technologies have evolved from simple methods of concealing information into complex systems that encompass digital signatures, electronic certificates, and multi-layered security protocols.

## 1-2. The Research Problem

Security has become a critical factor in the digital economy, within which e-commerce constitutes an integral part. However, the primary challenge facing online business today is its susceptibility to breaches and security attacks, such as data theft and information leakage. Sources confirm that fraud and cyberattacks can occur with relative ease during online transactions.

The threat of cyberattacks has become increasingly sophisticated, jeopardizing business continuity and the integrity of customer data. Cyberattacks—including malware infections and data breach attempts—have resulted in catastrophic losses and significant damage to corporate reputations.

- **Loss of Trust in Centralized Systems:** Weak or inadequate security in traditional systems leads to a loss of customer trust, which is critical for maintaining business continuity. Compromised consumer confidence negatively impacts sales and business growth. The original motivation behind the creation of Blockchain, as introduced in the Bitcoin whitepaper, was to produce a virtual currency that could be traded between two parties without the need for a third party to act as an intermediary or a transaction validator. Blockchain operates on a decentralized network that eliminates the need for intermediaries, thereby reducing the risk of a Single Point of Failure (SPOF).
- **Data Integrity and Tampering Issues:** In traditional systems, altering records is possible. Blockchain, however, provides a solution to this through its immutability, ensuring that recorded data cannot be changed. This guarantees data integrity and makes it virtually impossible for attackers to modify or compromise sensitive information without authorization.
- **The Need for Reliable Authentication:** In online transactions, verifying user identity is essential to prevent fraud. This has necessitated the use of

Blockchain to create secure, encrypted digital identities that can be utilized for robust identity verification and authentication.

- **Lack of Transparency and Transaction Traceability:** In traditional supply chains, tracking products and transactions has consistently proven difficult. However, **Blockchain** provides full transparency, as every transaction is recorded, visible, and traceable to all authorized users. This level of transparency helps prevent fraud and fosters more secure transactions.

### 1-3. Research Objective

Cybersecurity is a fundamental pillar for the sustainability of **Online Business**. With the rapid expansion of the global digital ecosystem, the complexity of cyber threats—which jeopardize business continuity and customer data integrity—is also increasing. The primary problem facing online businesses today is their susceptibility to security attacks, such as data theft and information leakage. Failure to effectively address these security challenges leads to severe consequences, including loss of customer trust, financial losses, and irreparable reputational damage. To confront these escalating challenges, **Blockchain** technology emerges as an innovative solution.

Given that cyber threats are characterized by continuous evolution, the objective focuses on how to implement and optimize **Blockchain** technology as an effective security solution. This aims to enhance the **cyber resilience** of online businesses in the face of an ever-evolving threat landscape.

### 1-4. Technical Challenges

#### 1-4.1. Network Latency & Consensus Overhead

In consensus-based systems, processing latency increases proportionally with the increase in the number of network nodes.

- **The Challenge:** The system requires approval from more than 50% of the total node weights before accepting any new block. In traditional networks, this results in significant node-side delays (blocking) as the system waits for all HTTP requests sent to peer nodes to complete, which may lead to UI freezing.
- **Implemented Engineering Solution:** I addressed this issue within the **NodeCommunicationService.cs** file by adopting an Asynchronous Programming model using Task and HttpClient. This approach enables the system to broadcast transactions and receive votes in the background, ensuring that core business operations within the shopping cart remain non-blocking and uninterrupted.

#### 1-4.2. Digital Key Management & Security

The entire security of the system relies on the confidentiality of the **Private Key**. If this key is compromised, an attacker could sign malicious blocks and impersonate a trusted node.

- **The Challenge:** In development environments, configurations and keys are often stored in plaintext files, such as **appsettings.json**. This poses a significant security risk if the source code is uploaded to public repositories or if the server is compromised.
- **Implemented Engineering Solution:** I implemented a dynamic key generation system within **KeyGeneratorService.cs** that generates a new 2048-bit key pair upon node startup. Furthermore, Public Key Infrastructure (PKI) principles were applied; only the public key is stored on the blockchain for transparent access, while the private key remains hidden and is utilized exclusively within the node's secure memory for message signing.

#### 1-4.3. Chain Forks & Data Synchronization

In distributed networks, connectivity between nodes may be interrupted for several seconds, leading to the existence of divergent versions of the **blockchain** across different nodes.

- **The Challenge:** Ensuring that all nodes (Node A, B, and C) maintain a Single Source of Truth without conflicts. If Node A appends a new block while Node B is offline, a discrepancy will occur in the Previous Hash values, leading to a loss of synchronization across the network.
- **Implemented Engineering Solution:** I implemented a chain replacement algorithm (ReplaceChain) within **BlockchainService.cs**. This algorithm is based on the "Longest Valid Chain" rule; the node automatically compares its local chain with those of its peers. If a longer chain with valid digital signatures is identified, the node immediately replaces its local chain to ensure full data synchronization across the network.

#### 1-4.4. Sybil Attack & Weight Manipulation

There is a risk of an attacker creating numerous nodes under pseudonymous identities to attempt to seize control of the voting process (known as a Sybil Attack).

- **The Challenge:** Distinguishing between a legitimate node and a malicious (Sybil) node created solely for voting manipulation.
- **Implemented Engineering Solution:** A hybrid architecture combining centralized and decentralized solutions was integrated; a node can only join the network after obtaining a Secret Join Card (SJC) from the Authentication Server. Furthermore, the project implemented a "Node Age" mechanism within the weight calculation formula in **NodeService.cs**. In this system, new nodes are assigned a negligible weight that does not impact the consensus decision. Trust and weight are then acquired incrementally over time, contingent upon the node's stability and persistence within the network.

### 1-5. Literature Review

- The researchers Zhaofeng et al[1]. proposed in this paper a blockchain-based decentralized authentication modeling scheme called (BlockAuth) designed for edge and IoT environments. This method contributes to providing a strong fault-tolerance solution by regarding each edge device as a node to form a blockchain network.

The method aims to secure registration and authentication processes in distributed environments, where a specialized consensus protocol and smart contracts were developed to ensure these operations are executed automatically and securely. This approach relies on a decentralized architecture that prevents single points of failure and increases the reliability of the nodes joining the network.

- The researchers Moinet et al[2]. proposed in this paper a comprehensive solution based on blockchain technology for authentication and trust evaluation in decentralized ad-hoc sensor networks. This method contributes to addressing serious security issues in self-organized and evolving networks. The study aims to leverage the "immutability" feature of the blockchain to ensure the integrity of identity data and mutual trust evaluations between devices. The method adopts a mechanism to continuously evaluate the behavior of nodes within the network, contributing to the creation of a "Web of Trust" that prevents malicious nodes from accessing network resources or disrupting its functions.
- The researchers Shi et al[3]. proposed in this paper an advanced blockchain-empowered authentication, authorization, and auditing (AAA) strategy to ensure data security in large-scale heterogeneous networks (LS-HetNet). This strategy contributes to redefining digital identity by using blockchain account addresses as unique identifiers during the authentication process. The method aims to protect data privacy and prevent unauthorized access by redesigning the access control authority and storing it directly on the blockchain. This approach relies on the principles of transparency and traceability of operations, which reduces the risks of both internal and external attacks in big data environments.
- The researchers Al Ahmed et al[4]. proposed in this paper an innovative approach for decentralized authentication by using blockchain technology, where the authentication process is optimized by organizing IoT devices into clusters based on their computational capability, energy reserve, and location. The method aims to manage the authentication process through a hierarchical structure of interconnected blockchains, in which each cluster undergoes authentication independently. The study relies on a proposed consensus protocol to reduce processing load, based on verifying the identity-based encryption key signature of the device and its corresponding cluster, ensuring high efficiency in distributed IoT environments.
- The researcher Hong proposed in this paper[5] an IoT sensor node authentication scheme based on peer-to-peer (P2P) networking and blockchain technology. This method contributes to providing integrity and non-repudiation functions in addition to the core authentication function. The study aims to build a secure and lightweight authentication scheme suitable for the limited resources of IoT devices, where blockchain is used as a

distributed ledger to ensure node reliability and tamper-proof data exchange. This approach relies on decentralization to enhance security in sensor networks operating in open environments.

## 1-6. State-of-the-art (SOTA)

The field of distributed network security and blockchain has witnessed immense advancements recently. The focus has shifted from merely securing financial transactions to building intelligent authentication and authorization systems capable of countering sophisticated cyberattacks.

### 1-6.1. Distributed Zero Trust

Modern systems no longer grant permanent trust to any node or user upon network entry; instead, the field has shifted toward the concept of Continuous Authentication.

**Behavior-Based Authorization:** Node activity is monitored in real-time, and permissions are dynamically adjusted based on an ongoing assessment of the node's trust level[6].

### 1-6.2. Proof of Reputation (PoR)

Recent research has transcended the energy-intensive Proof of Work (PoW) algorithm, shifting toward lighter and more intelligent consensus mechanisms.

**Weighted Voting:** Nodes with a long history of secure behavior are granted higher trust weights, making their votes more influential in the validation and acceptance of new transactions[7].

### 1-6.3. Edge Computing

To mitigate latency issues, current research is shifting toward processing authentication at the network edge rather than routing it to a centralized cloud.

**Blockchain-Edge Synergies:** Security tasks are distributed across nodes in close proximity to the end-user. This provides real-time responsiveness while maintaining the highest standards of decentralized security[8].

### 1-6.4. Sybil Resistance

Recent studies have focused on protecting distributed networks from pseudonymous nodes (Sybil nodes) that attackers attempt to create to seize control of the voting process.

**Multi-layered Verification Mechanism:** Integrating RSA digital keys with trust weights based on "Node Age" to prevent any untrusted new nodes from compromising the system's security[9].

## 1-7. Conclusion

In conclusion, this chapter has laid the foundational framework for the research by highlighting the pivotal role of information security and encryption in the modern digital era. The discussion underscored how traditional centralized systems struggle with critical vulnerabilities, such as Single Points of Failure (SPOF) and data integrity

issues, which can severely compromise customer trust and business continuity. Blockchain technology emerged as a transformative solution, offering a decentralized, immutable, and transparent architecture to address these persistent challenges.

Furthermore, the chapter detailed the specific technical hurdles encountered during the engineering phase—including network latency, digital key management, and synchronization issues—and presented the implemented solutions such as asynchronous programming and dynamic 2048-bit RSA key generation. By reviewing the current literature and the state-of-the-art developments in Proof of Reputation (PoR) and Edge Computing, it is evident that the field is moving towards more intelligent and resilient authentication schemes.

## **Chapter 2: Theoretical Framework**

## 2-1. Introduction

This chapter establishes the **theoretical framework** for **blockchain technology**. It provides an overview of its fundamental concepts and its various **consensus mechanisms**. Furthermore, it explores potential **vulnerabilities** and **breaches** that may target blockchain systems, along with the **information security measures** implemented to safeguard the technology.

## 2-2. Blockchain Technology

### 2-2.1. The Fundamental Concept of Blockchain

The technical concept of Blockchain first emerged in 2008 in the white paper published by Satoshi Nakamoto [10]. It was later practically implemented in 2009 with the launch of Bitcoin.

Blockchain technology is defined as a Distributed Ledger Technology (DLT) that records transactions in a series of blocks linked sequentially and chronologically using cryptography. This process creates an immutable and tamper-proof record. The fundamental objective of this technology is to establish an electronic payment system based on cryptographic proof rather than reliance on a central intermediary. This architecture allows any two parties to transact directly with one another without the need for a trusted third party [11].

Initially, Blockchain 1.0 emerged with the advent of cryptocurrencies, specifically Bitcoin. Subsequently, Blockchain 2.0 introduced the concept of Smart Contracts, expanding the scope of application beyond simple financial transactions. Blockchain 3.0 facilitated technical applications across diverse sectors, such as healthcare. Blockchain 4.0 focused on real-time technological integration, while Blockchain 5.0 explores the synergy between Blockchain and Artificial Intelligence (AI) to develop decentralized finance and Web3 applications [11], as illustrated in Figure (1-1).

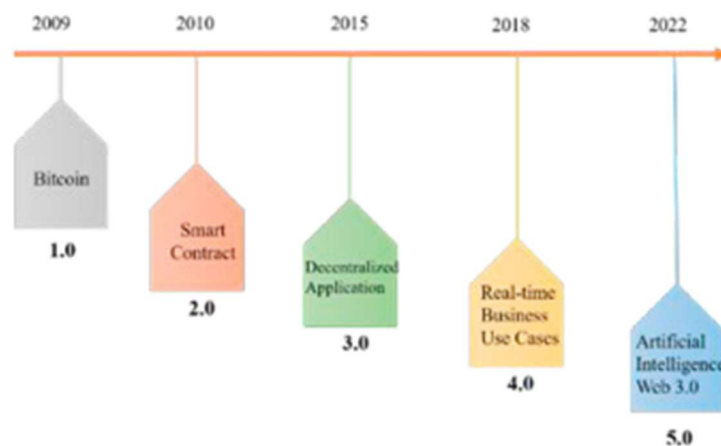


Figure 1 : Blockchain Technology Generations

#### 2-2.1.1. The Architecture of Blockchain

Blockchain technology provides a distributed and decentralized method for storing data records. It relies on a linked chain of blocks that store validated transactions. Each block contains a cryptographic hash, serving as a unique digital fingerprint that ensures data integrity, and a distributed ledger that links these blocks together into an integrated, encrypted chain.

The first block in a blockchain is known as the Genesis Block [11]. Starting from the second block, each subsequent block stores the hash value of the preceding block.

The Block Header consists of several essential elements:

- **Block Version (4 bytes):** Defines the specific rules and protocols used for block validation.
- **Merkle Root Hash (32 bytes):** Represents the consolidated digital fingerprint of all transactions within the block.
- **Previous Block Hash (32 bytes):** The cryptographic link that connects the block to its predecessor, forming the basis of the immutability principle.
- **Timestamp (4 bytes):** Records the exact time the block was created.
- **Nonce (4 bytes):** A unique random value adjusted by miners to generate a hash that meets the predefined difficulty target.

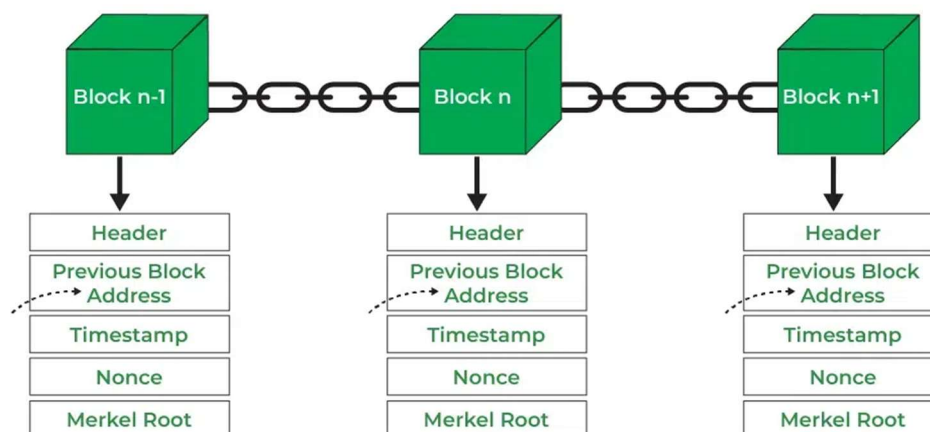


Figure 2 : The Fundamental Architecture of a Blockchain Network

### 2-2.1.2. Blockchain Workflow

Blockchain users are referred to as Nodes, and each node within the network maintains a complete copy of the blockchain ledger. Certain nodes are designated as Miners; these nodes possess specialized capabilities that allow them to validate transactions and append new blocks to the network [11] in accordance with the established consensus mechanism.

These nodes utilize public and private keys to sign transactions, which can range from fund transfers between two nodes to any type of peer-to-peer contract. Utilizing the Gossip Protocol, all transactions are broadcast to every node in the network. Subsequently, miners verify and authenticate these transactions before incorporating them into a block. This entire process is known as Mining [11].

### 2-2.2. Consensus Mechanisms

Consensus mechanisms constitute the technical foundation of the blockchain network, providing it with trust and security. They can be defined as the set of rules that facilitate network protection against malicious behavior and security attacks. Through these mechanisms, nodes reach an agreement on the state of the distributed ledger and the validity of transactions without the need for a central intermediary [12].

These mechanisms primarily differ in terms of energy requirements, security levels, and scalability. The most prominent of these mechanisms include:

- Proof of Work (PoW) is the first consensus mechanism to be adopted, and it was pioneered by the Bitcoin network. The PoW system allows all miners on the network to participate in adding new blocks by competing to solve highly complex mathematical puzzles. The "winning" miner is the first to compute a 64-digit hexadecimal number, known as the hash, and is rewarded by appending the new block to the chain [12]. While this mechanism is considered the most reliable and secure to date, it requires substantial operational costs.
- In Proof of Stake (PoS), validators are selected by depositing a portion of their cryptocurrency as collateral [12] to be randomly chosen as validators. This model is significantly more environmentally friendly, as the miner's computational power is no longer the primary factor. It is evident that Proof of Stake is more profitable for participants holding a larger number of tokens, which may lead to centralization—a generally undesirable outcome in blockchain networks.
- Delegated Proof of Stake (DPoS) is an evolution of the PoS concept, where candidate nodes are selected, and stakeholders are granted the right to vote for suitable nodes that possess the necessary credentials to generate new blocks. This is done in exchange for staking a portion of their tokens, with the voting weight determined by the size of the stake. After transaction validation, the elected nodes are rewarded, and they, in turn, share the reward with the voters who supported them.
- Other systems rely on Proof of Authority (PoA), where miners are selected based on their reputation ratings. In the PoA algorithm, validators do not rely on staked cryptocurrencies; instead, they leverage their reputation to obtain the necessary rights to validate blocks.

**Table 1 : An Analytical Comparison of Key Consensus Mechanisms**

| Criteria / Mechanism           | PoW                              | PoS                           | DPoS                          | PoA                         |
|--------------------------------|----------------------------------|-------------------------------|-------------------------------|-----------------------------|
| Core Principle                 | Computational Power              | Economic Stake (Value)        | Voting and Delegation         | Reputation and Identity     |
| Decentralization Degree        | Very High                        | High                          | Medium to Low                 | Very Low (Semi-centralized) |
| Energy Consumption             | Very High                        | Very Low                      | Very Low                      | Low                         |
| Transaction Speed (Throughput) | Low (3-7 TPS)                    | Medium to High (100–100,000*) | Very High (Thousands)         | High                        |
| Security Vulnerabilities       | 51% Attack                       | 51% Stake Attack              | Miner Collusion / DoS Attacks | Miner Collusion             |
| Governance Structure           | Decentralized (Based on Compute) | Decentralized (Wealth-based)  | Partially Democratic          | Full Centralization         |

### 2-2.3. Smart Contracts

#### 2-2.3.1. The Technical Foundations of Smart Contracts

According to contemporary literature, Smart Contracts are defined as computer programs that automatically execute contractual terms without the need for an intermediary, based on pre-defined conditions [13]. These contracts function by embedding terms and conditions within a source code that is deployed onto the blockchain network.

The concept of Smart Contracts first emerged in the 1990s within the pioneering work of Szabo [14], where the core concept was defined as a set of promises specified in digital form, including protocols through which parties fulfill these contracts. With the advent of blockchain technology, the possibility of bringing this concept into practical implementation was realized.

Smart contracts possess several key advantages, most notably: Autonomy, as once a smart contract is deployed, it becomes capable of automatic execution; Decentralization, where the contract is distributed across the blockchain network rather than a central server; Self-executability, meaning the contract not only defines the rules but also enforces them; Determinism, where the contract's outcome remains consistent as long as it is executed with the same inputs; and one of the most critical features is Transparency, as the contract and its records are typically accessible to all nodes within the blockchain network[13].

### 2-2.3.2. The Operational Mechanism of Smart Contracts with Decentralized Applications (DApps)

The decentralized application (DApp) sends a request to the blockchain network to access the smart contract. The network authenticates the request and grants the application access rights. The application then interacts with the contract by invoking its functions; the contract processes the sent request and executes the appropriate action. Finally, the application updates the user interface to display the interaction result to the user [15].

The smart contracts are how Solidity's code is contained. Therefore, a contract in Solidity is a group of functions and state-related data stored at a particular address on the Ethereum blockchain. The entire quantity of cryptocurrency that can be withdrawn for each entity can be divided using specific functions that can be placed into smart contracts. Wallets are frequently used to store cryptocurrencies; however, in this framework, smart contracts are employed instead because they interact directly with dApp. Therefore, they are safer to use to store cryptocurrencies than wallets. Hacking into cryptocurrency wallets is possible, but it is impossible to hack into smart contracts[15].

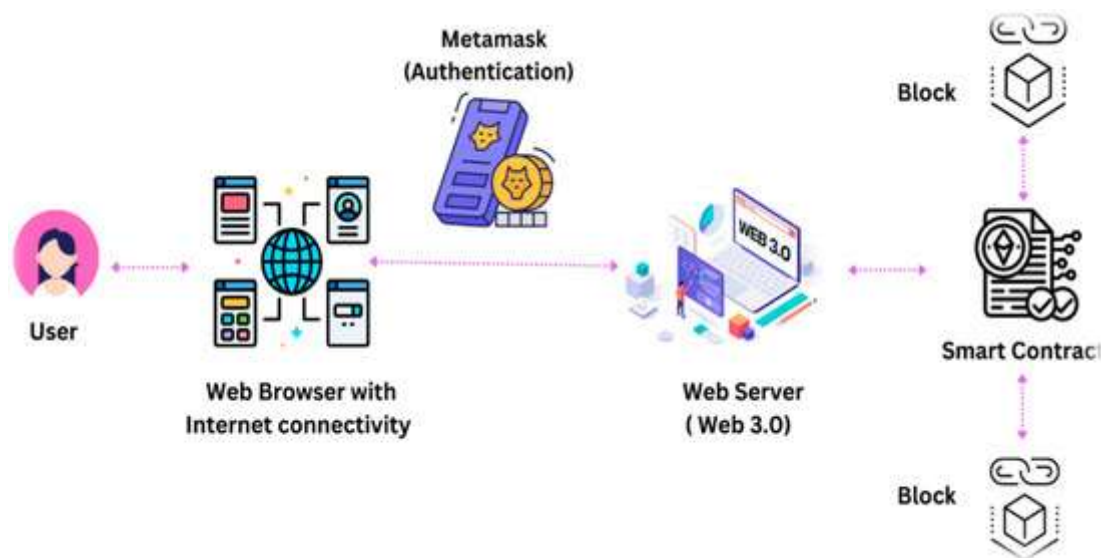


Figure 3 : Interaction of DApps and Smart Contract for Consumer Electronics.

### 2-2.4. Security Breaches and Attacks on Blockchain Technology

Blockchain technology is considered an emerging service framework that integrates cryptography, peer-to-peer (P2P) networks, consensus mechanisms, and smart contract technology. It possesses key characteristics such as decentralization, immutability, and transaction traceability. However, the expansion of application scenarios and the increasing complexity of attack techniques have seriously threatened the security of the blockchain system.

### 2-2.4.1. Major Attacks in Blockchain Technology

Table 2 : The Most Representative and Common Attacks on Blockchain

| Attack Type            | Principle and Impact                                                                                                                                                                                                                                                                                                                                        |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 51% Attack             | An individual or group gains control over more than 50% of the network's computing power (hashrate). This allows them to manipulate the transaction ledger, such as reversing transactions, performing double-spending, and preventing other nodes from mining new blocks, which undermines the integrity of the blockchain[16].                            |
| Double-Spending Attack | The act of using the same digital currency unit more than once, which destroys the properties of immutability and consistency in the blockchain. Attackers exploit delays and vulnerabilities in the consensus mechanism to broadcast two different transactions simultaneously, where one is confirmed while the other is hidden or replaced[17].          |
| Re-entrancy Attack     | A common vulnerability in Smart Contracts. It occurs when a contract fails to update its state (e.g., balance update) correctly before completing a fund transfer. The attacker exploits the time gap between execution logic and state update to recursively call the withdrawal function multiple times before the initial execution is finished[16][18]. |
| Replay Attack          | Involves the duplication or delay of valid data. Attackers exploit the transparency of transactions to intercept a previously signed transaction and re-broadcast it to the network. This leads to the transaction being executed multiple times on the same chain or different forks, resulting in loss of funds or resources.                             |
| Sybil Attack           | A typical identity spoofing pattern in self-organizing network environments. Attackers infiltrate the system by creating multiple fake identities (nodes) to exploit size advantages, gain control over resources, and disrupt the                                                                                                                          |

|                                                 |                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                 | consensus mechanism, which can lead to a complete denial of service[16].                                                                                                                                                                                                                  |
| Timestamp Tampering Attack / Time-Bandit Attack | Exploiting the fluctuation and inaccuracy of timestamps in the blockchain system (usually determined by miners) to manipulate mechanisms or conditions that depend on time within Smart Contracts. Attackers can trigger or delay specific contract functions before their intended time. |

#### 2-2.4.2. Security Threats Classified by Blockchain Layers

Threats are analyzed based on the six-layer blockchain architecture, where each layer encounters unique security challenges.

##### 1- Data Layer

This layer relies on hashing and Merkle trees to ensure data integrity and prevent tampering. The security threats include:

- **Quantum Computing Attacks:** These pose a significant challenge, as algorithms such as Shor's algorithm can break traditional cryptographic algorithms like ECDSA and RSA used in digital signatures. Additionally, Grover's algorithm reduces the time complexity for finding hash collisions[16].
- **Hash Collision Attacks:** Exploiting vulnerabilities in hashing algorithms to allow multiple distinct inputs to generate the same hash value. This leads to a degradation in the performance of hash tables and may trigger a Denial of Service (DoS) attack[16].

##### 2- Network Layer

This layer operates via a Peer-to-Peer (P2P) network architecture. The most prominent attacks include:

- **Denial-of-Service (DoS) Attacks:** Transactions are blocked or delayed by exhausting the network's computational or bandwidth resources[16].
- **Partition Attacks:** Dividing the network into isolated segments, which disrupts the consensus mechanism and leads to the formation of unintended forks[16].
- **Eclipse Attacks:** Attackers gain control over all neighboring nodes of a targeted node to isolate it from the rest of the network, forcing it to receive fraudulent or manipulated data[16].

##### 3- Consensus Layer

This layer is susceptible to attacks aimed at controlling the decision-making process. The most prominent threats include:

- **51% Attack:** Gaining control over the majority of the network's total computing power (hashrate)[16].

- **Sybil Attack:** Forging multiple fake identities to dominate the consensus process and manipulate outcomes[16].
- **Selfish Mining Attacks:** Withholding newly discovered blocks to build a longer private chain and gain an unfair advantage[16].
- **Pre-computation Attacks:** In Proof of Stake (PoS) mechanisms, an attacker predicts and monopolizes block production rights[16].

#### 4- Incentive Layer

This layer pertains to the economic mechanisms and reward systems of the blockchain. The primary security threat includes:

- **Block Withholding Attacks:** A malicious behavior where a miner intentionally withholds the valid hash of a newly mined block rather than broadcasting it to the network. The objective is to secretly build a longer private chain to gain an unfair advantage and secure higher rewards at the expense of the network's integrity[16].

#### 5- Smart Contract Layer

This layer involves security threats arising from flaws within the source code. The most prominent vulnerabilities include:

- **Reentrancy Attacks:** Exploiting logical vulnerabilities in the contract's code, as previously mentioned, to recursively call functions before state updates[16].
- **Timestamp Dependency Attacks:** Manipulating contract mechanisms that rely on block timestamps for execution logic[16].
- **Integer Overflow/Underflow:** Defects in the arithmetic logic of the code regarding the storage of numerical values[16].
- **Denial of Service (DoS) Attacks:** Exhausting Gas resources through malicious transactions to halt or disrupt contract operations.

#### 6- Application Layer

This layer represents the final user interaction point. The security threats include:

- **Social Engineering Attacks:** Enticing users to reveal sensitive information, such as private keys, through phishing websites or fraudulent emails[16].
- **Front-end Interface Hijacking and Code Injection:** Manipulating web pages to plant malicious code or tamper with user transactions[16].
- **Data Breach:** Intercepting transmitted data that is either unencrypted or weakly encrypted[16].
- **Improper Management of Permissions:** Failures in access control that may lead to unauthorized operations and transactions[16].

### 2-2.4.3. Security Overview and Historical Threats

Table 3 : Examples of Major Security Incidents During the Evolution of Blockchain

| Date          | Incident                                                              | Impact / Approximate Loss                                                                    |
|---------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| February 2014 | Mt. Gox Exchange Hack (The largest Bitcoin exchange at the time)[16]. | Theft of 850,000 BTC, with an estimated value of approximately \$500 million USD[16].        |
| June 2016     | The DAO Project Exploit (Ethereum Ecosystem)[18][16].                 | Theft of approximately 3.6 million ETH[18][16].                                              |
| August 2021   | Poly Network Cross-Chain Protocol Hack.                               | Theft of crypto assets valued at \$610 million USD.                                          |
| March 2022    | Ronin Network Hack (Blockchain Gaming Project).                       | Theft of 173,000 ETH and 25.5 million USDC, with a total loss approaching \$625 million USD. |
| May 2024      | DMM Bitcoin (Japan) Hack.                                             | Theft of 4,029 BTC, leading to a loss exceeding \$300 million USD.                           |

## 2-2.5. Information Security Measures in Blockchain Technology

### 2-2.5.1. Fundamental Security Measures Derived from Cryptography

Blockchain systems rely fundamentally on cryptographic techniques to ensure data security and integrity. The foundational security measures based on cryptography include the following:

- 1- **Hash Functions:** Used to ensure data integrity and immutability. The SHA-256 algorithm is the primary hashing algorithm used for linking blocks, constructing Merkle trees, and verifying data integrity[17].
- 2- **Digital Signatures:** Implemented via asymmetric cryptography, digital signatures are essential for verifying transaction legitimacy and data integrity. They ensure identity authentication and non-repudiation. To mitigate double-spending attacks, the ECDSA (Elliptic Curve Digital Signature Algorithm) is most commonly used due to its high efficiency compared to traditional algorithms like RSA[17].
- 3- **Data Encryption**
  - **Symmetric Encryption:** Used to protect the confidentiality of sensitive data due to its high speed in processing large volumes of data[16].

- **Asymmetric Encryption:** Used to implement security functions such as identity authentication and digital signatures[16].
- **Homomorphic Encryption:** Addresses privacy issues regarding sensitive data during the execution of smart contracts, as it allows for direct computation on encrypted text (ciphertext)[16].
- 4- **Zero-Knowledge Protocols (ZKP):** Used to protect privacy in blockchain, these protocols enable a prover to demonstrate the validity of information without revealing its underlying content. zk-SNARKs are the predominant cryptographic tool used for private Ethereum transactions and protecting the privacy of smart contracts[16].
- 5- **Post-Quantum Cryptography (PQC):** Aims to develop algorithms resistant to quantum computing attacks, such as Shor's algorithm, which threatens RSA and ECC. Its applications in blockchain include transaction signatures, data encryption, and identity verification. Examples of quantum-resistant algorithms include CRYSTALS-Kyber, FALCON, and SPHINCS+.
- 6- **Public Key Infrastructure (PKI):** Essential for identity authentication, data encryption, and signature verification; it ensures decentralized trust and prevents malicious nodes from joining the network.

#### 2-2.5.2. Security Measures by Blockchain Layers

Table 4 : Defensive Measures for Each Blockchain Layer

| Blockchain Layer | Proposed Security Measures for Mitigation                                                                                                                                                                                                                                                                                                                                          |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data Layer       | Countering quantum computing attacks by implementing Post-Quantum Cryptography (PQC) algorithms such as CRYSTALS-Kyber (for encryption) and FALCON or SPHINCS+ (for digital signatures). Resisting hash collision attacks by using hash functions with stronger collision resistance and improving hash table collision resolution mechanisms (e.g., open addressing or chaining). |
| Network Layer    | Countering Denial-of-Service (DoS) attacks through traffic monitoring or Proof of Work (PoW). Preventing Partition attacks via multi-path routing and dynamic node adjustments. Mitigating Eclipse attacks by using diverse peer pools and reputation-based mechanisms.                                                                                                            |
|                  | Resisting Sybil attacks by establishing rigorous identity authentication for nodes (including certificates from trusted nodes) and increasing the cost of creating multiple identities. Resisting                                                                                                                                                                                  |

|                   |                                                                                                                                                                                                                                                                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Consensus Layer   | Selfish Mining through Fork detection mechanisms. Resisting Pre-computation attacks in Proof of Stake (PoS) using Verifiable Delay Functions (VDF), dynamic stake mitigation, and forward-secure hash chains.                                                                                                                                 |
| Incentive Layer   | Preventing Block Withholding Attacks through real-time operational monitoring systems and implementing disciplinary incentive mechanisms (slashing) to eliminate malicious miners.                                                                                                                                                            |
| Contract Layer    | Countering Reentrancy attacks through code auditing (e.g., updating balances before transfers) or using mutex locks. Utilizing trusted third-party time sources instead of blockchain timestamps to mitigate dependency issues. Countering DoS attacks by establishing multi-signature governance mechanisms and granular permission control. |
| Application Layer | Implementing End-to-End Encryption (E2EE) for data transmission, conducting regular security audits, utilizing Multi-Signature (Multi-Sig) mechanisms, and establishing security awareness programs for users.                                                                                                                                |

### 2-2.5.3. Design and Implementation of Specific Defensive Solutions

**Table 5 : Design and Implementation of Innovative Defensive Solutions Using Smart Contracts**

| Attack Type            | Designed Security Measure (Solution Principle)                                                                                                                                                                                                                                                                                                                |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 51% Attack             | Implementing a Historical Weighted Difficulty (HWD) mechanism. The branch weight is calculated based on the frequency of block generation by miners within a historical window. This ensures that an attacker who temporarily increases their computing power will find their attack chain's weight remains low, preventing the network from switching to it. |
| Double-Spending Attack | Introducing Payment Status and network confirmation mechanisms. Users must wait for network block confirmations (e.g., after 24 blocks) before a payment is confirmed. No new payment initiation                                                                                                                                                              |

|                    |                                                                                                                                                                                                                                                                                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    | is permitted if a previous payment remains To Be Confirmed (TBC).                                                                                                                                                                                                                                                                                       |
| Reentrancy Attack  | Utilizing Combined Mutual Exclusion Locks. This combines dynamic and hierarchical locks, where a unique mutex is assigned to each account. Critical operations (such as withdrawals) are only executed if permitted by both locks simultaneously.                                                                                                       |
| Replay Attack      | Incorporating a unique Nonce value and a Transaction Validity Period. A unique transaction identifier (txHash) is generated using the sender's address, amount, Nonce, and timestamp. This identifier is verified to ensure the transaction has not already been processed.                                                                             |
| Sybil Attack       | Building upon the PBFT mechanism, a Reputation Value is assigned to each node. The network's consensus state is only updated when the sum of reputation values from agreeing nodes is sufficient and exceeds a specific threshold (Rts), thereby mitigating the impact of malicious nodes.                                                              |
| Time-Bandit Attack | Utilizing a hybrid random number generation scheme that does not rely exclusively on timestamps. A Request ID is generated via a hash function using the responsible official's information, timestamp, and user address. This is then combined with block data (previous block hash and difficulty) to produce a final, non-predictable random number. |

## 2-3. Conclusion

In conclusion, this chapter has established the essential theoretical framework for blockchain technology by reviewing its historical evolution and its decentralized, immutable architectural foundations. The analysis demonstrates that the security of blockchain systems relies not only on robust consensus mechanisms but also on its ability to adapt to evolving threats through advanced cryptographic measures. However, the emergence of novel challenges—such as quantum computing and smart contract vulnerabilities—necessitates a shift from conventional methods toward the innovation of specific, intelligent defensive solutions. A deep understanding of these architectural layers and historical breaches serves as the primary reference for this research in the following chapters.

## **Chapter 3: Setting Up the Development Environment**

### 3-1. Introduction

Preparing a secure and stable development environment is considered one of the most critical stages in cybersecurity and blockchain-based systems, especially when the platform depends on distributed communication, blockchain validation, cryptographic operations, and secure transaction processing. The development environment used in this project was not designed only to support traditional web application development, but also to provide a decentralized infrastructure capable of simulating real blockchain network behavior, secure node synchronization, and distributed trust management.

The proposed system represents a blockchain-based e-commerce security platform that combines modern web technologies with advanced information security concepts to ensure secure transaction handling and reliable data verification. Unlike traditional centralized systems, the implemented environment supports distributed blockchain communication between multiple independent nodes, where each node participates in transaction validation and blockchain synchronization.

The architecture of the environment integrates several security-oriented mechanisms including:

- Distributed trust management
- Blockchain-based validation
- Cryptographic key generation
- Secure peer-to-peer communication
- Transaction integrity verification
- Audit logging and monitoring
- Multi-layer request validation
- Distributed node synchronization

To achieve these objectives, the development environment integrated multiple software frameworks and technologies including ASP.NET Core, Entity Framework Core, SQL Server LocalDB, blockchain service components, RESTful APIs, Razor Pages, and distributed node communication protocols.

The environment was configured to simulate a decentralized blockchain network composed of multiple independent nodes communicating securely with each other while maintaining synchronized blockchain records and distributed transaction histories. Each node operates independently with its own database instance, blockchain copy, configuration profile, and communication endpoint. This configuration enabled the project to emulate realistic distributed security environments similar to those used in modern blockchain platforms and decentralized systems.

In addition to blockchain functionality, the system environment also supports:

- MVC web architecture
- Service-oriented backend design
- Session-based transaction management
- Database abstraction through repositories

- Blockchain explorer interfaces
- Distributed validation workflows
- Secure API communication

The development environment was carefully structured to ensure:

- Scalability
- Maintainability
- Security
- Modularity
- Distributed fault tolerance
- Separation of concerns
- Secure communication between layers

This chapter explains the complete development and configuration environment used during the implementation phase of the project. The chapter covers software technologies, hardware requirements, blockchain architecture, database preparation, frontend and backend setup, distributed node configuration, peer-to-peer communication mechanisms, cryptographic services, API configuration, and security-oriented system architecture.

### 3-2. Hardware Requirements

The implementation and testing of the proposed blockchain-based platform required a development environment capable of supporting distributed node execution, database management, backend processing, and blockchain synchronization simultaneously. Since the project simulates multiple blockchain nodes communicating in real time, the hardware environment needed sufficient computational resources to ensure stable execution and synchronization performance.

The project was developed and tested using the following hardware specifications:

**Table 6 : Hardware Specifications of the Development Environment**

| Component           | Specification                 |
|---------------------|-------------------------------|
| Processor           | Intel Core i7 Processor       |
| RAM                 | 16 GB DDR4                    |
| Storage             | 512 GB SSD                    |
| Operating System    | Windows 11 64-bit             |
| Network Environment | Localhost Distributed Network |

The processor plays an important role in handling:

- Concurrent node execution
- Backend request processing
- Blockchain synchronization tasks
- Cryptographic operations

The use of 16 GB RAM allowed the environment to execute:

- Multiple blockchain nodes
- SQL Server LocalDB instances
- Visual Studio development environment
- API testing tools
- Background synchronization services

SSD storage improved:

- Database response speed
- Application startup time
- Migration execution performance
- Blockchain data access

The operating system environment was configured using Windows 11 because it provides stable support for:

- ASP.NET Core development
- SQL Server LocalDB
- Visual Studio integration
- Distributed localhost communication

The hardware environment therefore provided sufficient resources to simulate a distributed blockchain network while maintaining stable synchronization and transaction validation performance.

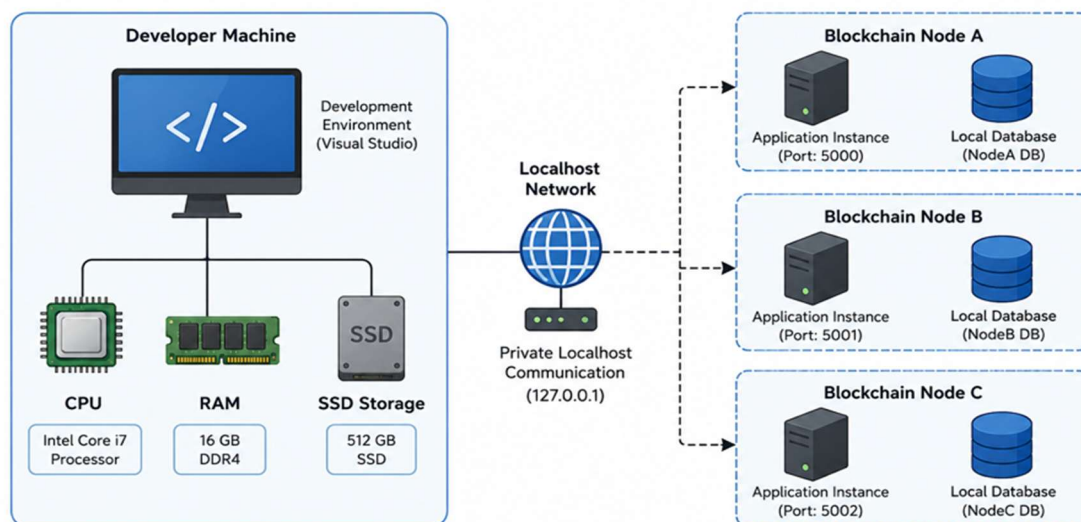


Figure 4 : Hardware Environment Diagram

### 3-3. Software Requirements

The proposed system required the integration of several software technologies and frameworks to support secure blockchain communication, distributed synchronization, database management, and web application development.

The development environment combined backend technologies, database systems, API frameworks, blockchain services, and frontend rendering technologies.

The following table summarizes the main software technologies used during implementation:

**Table 7 : The most important software technologies used**

| Software / Framework  | Purpose                            |
|-----------------------|------------------------------------|
| ASP.NET Core          | Backend web framework              |
| C#                    | Core programming language          |
| Entity Framework Core | ORM and database communication     |
| SQL Server LocalDB    | Relational database management     |
| Swagger               | API testing and documentation      |
| Visual Studio 2022    | Integrated development environment |
| Razor Pages           | Frontend rendering                 |

The backend environment was implemented using ASP.NET Core because it provides:

- High performance
- Secure middleware architecture
- RESTful API support
- Dependency injection
- Scalability

C# was selected because it supports:

- Object-oriented programming
- Asynchronous communication
- Cryptographic operations
- Distributed processing

Entity Framework Core simplified:

- Database interaction
- Migration management
- Entity mapping
- Query execution

SQL Server LocalDB was used because it provides lightweight relational database management suitable for local distributed blockchain simulation.

Swagger was integrated to provide:

- Interactive API testing
- Endpoint visualization
- Backend debugging
- Request validation

The frontend environment was implemented using Razor Pages and MVC architecture to provide reusable and dynamic user interfaces.

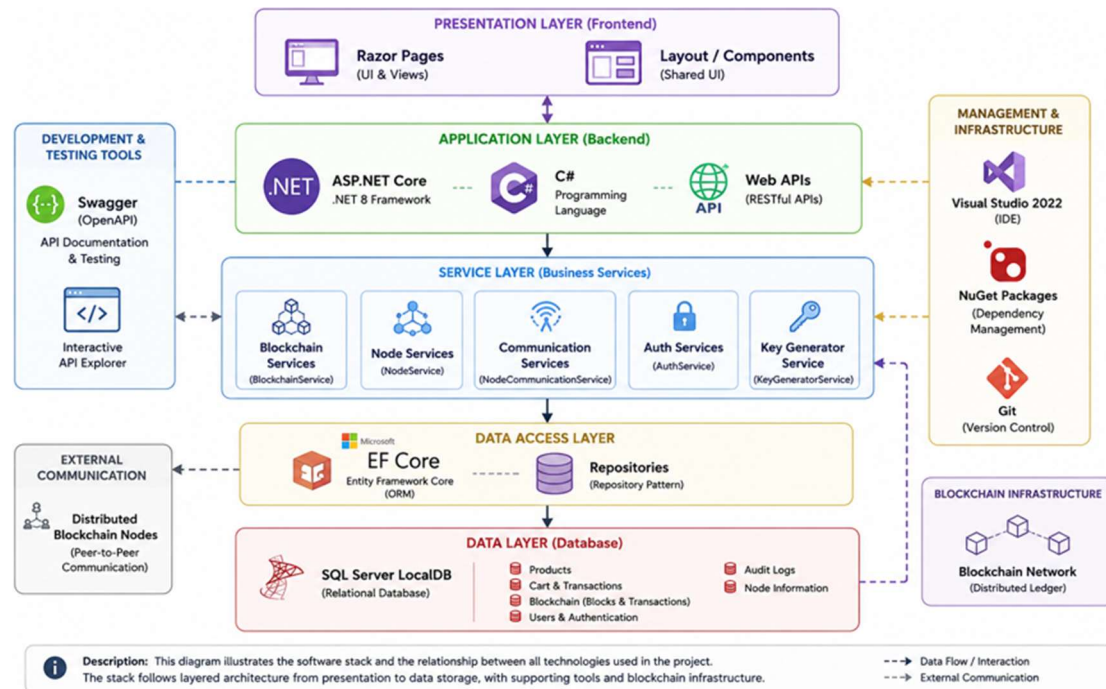


Figure 5 : Software Stack Diagram

### 3-4. Development Environment Setup

The development environment was configured carefully to support distributed blockchain execution, secure communication, database synchronization, and multi-node operation.

The setup process involved several stages including:

- Installing development tools
- Configuring backend frameworks
- Preparing databases
- Configuring blockchain nodes
- Setting up APIs
- Enabling distributed communication

The implementation process started by installing Visual Studio 2022 together with the .NET SDK required for ASP.NET Core development.

The environment additionally required:

- SQL Server LocalDB
- NuGet package dependencies
- Swagger configuration
- Entity Framework tools

NuGet packages were installed to support:

- API documentation
- Entity Framework Core
- OpenAPI integration
- Database migrations

The project configuration files were used to define:

- Application ports
- Environment variables
- Blockchain node identifiers
- Peer communication settings
- Database connection strings

The launch configuration supports multiple blockchain nodes including:

- Node-A
- Node-B
- Node-C

Each node operates using an independent localhost port:

- <http://localhost:5000>
- <http://localhost:5001>
- <http://localhost:5002>

The development environment was therefore capable of simulating decentralized blockchain communication entirely within the local machine environment.

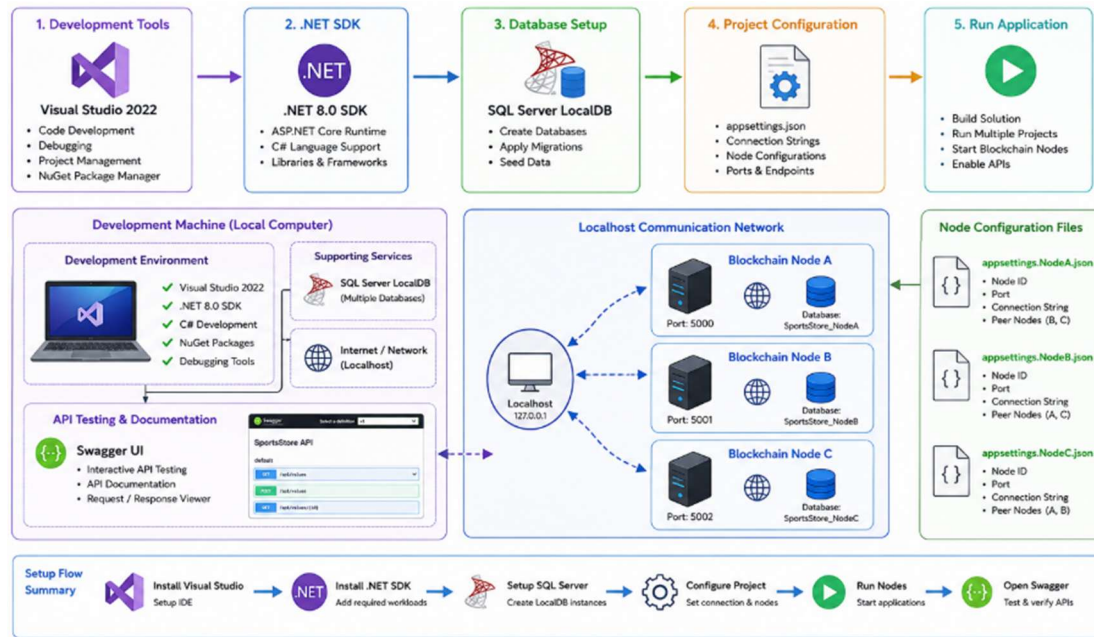


Figure 6 : Development environment setup flow diagram

### 3-5. Backend Environment Configuration

The backend environment of the proposed system was designed using a layered service-oriented architecture to support secure blockchain processing, distributed communication, transaction validation, and database management. The backend configuration represents the core operational environment responsible for handling all system logic and communication between different architectural layers.

The backend system was implemented using ASP.NET Core together with the MVC architectural pattern. The environment separates the application into several independent layers including:

- Controllers
- Services
- Repositories
- Database Context
- Blockchain Components

This separation improves:

- Maintainability
- Scalability
- Security
- Code reusability
- System organization

The backend environment processes:

- HTTP requests
- Blockchain validation
- Node synchronization
- Transaction management
- Database operations
- API communication

The project contains several backend controllers including:

- AuthController
- HomeController
- NodeController

These controllers are responsible for:

- Request routing
- Input validation
- Service communication
- Blockchain interaction
- API response handling

The backend environment also implements dedicated services including:

- BlockchainService
- NodeService
- NodeCommunicationService
- AuthService
- RegistryService
- KeyGeneratorService

Each service performs a specialized role within the system architecture.

The backend environment additionally integrates:

- Dependency Injection
- Middleware configuration
- REST API support
- Session management
- Distributed communication services

The implementation of service-oriented architecture reduced controller complexity by isolating blockchain and cryptographic logic into specialized backend services.

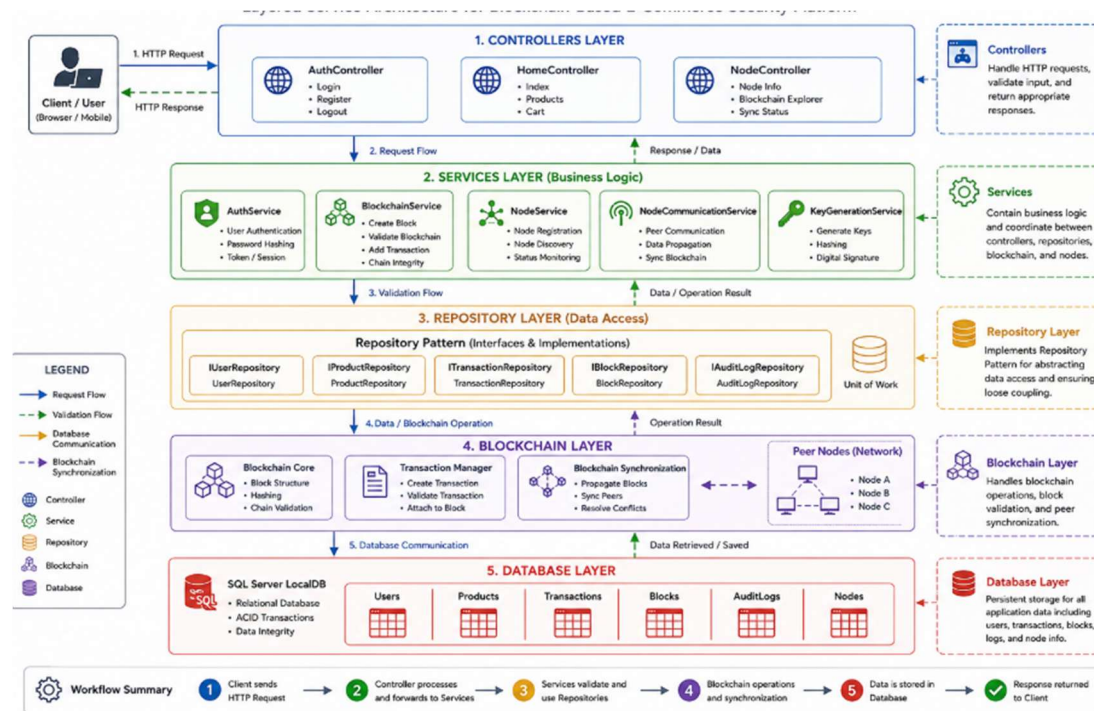


Figure 7 : Backend architecture configuration diagram

### 3-6. Database Environment Setup

The database environment was implemented using SQL Server LocalDB together with Entity Framework Core to provide reliable relational data storage and simplified database management.

The database subsystem stores:

- Blockchain records
- Transaction data
- Product information
- Shopping cart data
- Audit logs
- Node metadata

The project implements a dedicated database context called:

- StoreDbContext

The database context manages:

- Entity mapping
- Table generation
- Relationships
- Query execution
- Database transactions

Entity Framework Core migrations were used to automatically generate and update the database schema during development. The project includes:

- Initial migration files
- Database snapshots
- Seed data initialization

The migration architecture simplified:

- Database version management
- Schema updates
- Table generation
- Environment portability

The project additionally includes:

- SeedData.cs

which initializes the database with default records during startup.

Each blockchain node operates using an independent SQL database instance including:

- SportsStore\_NodeA
- SportsStore\_NodeB
- SportsStore\_NodeC

This distributed database architecture simulates decentralized blockchain environments where each node stores its own local blockchain copy and transaction history.

The database configuration is defined inside node-specific configuration files including:

- appsettings.NodeA.json
- appsettings.NodeB.json
- appsettings.NodeC.json

The database environment therefore supports:

- Distributed storage
- Blockchain replication
- Transaction persistence
- Audit logging
- Decentralized synchronization

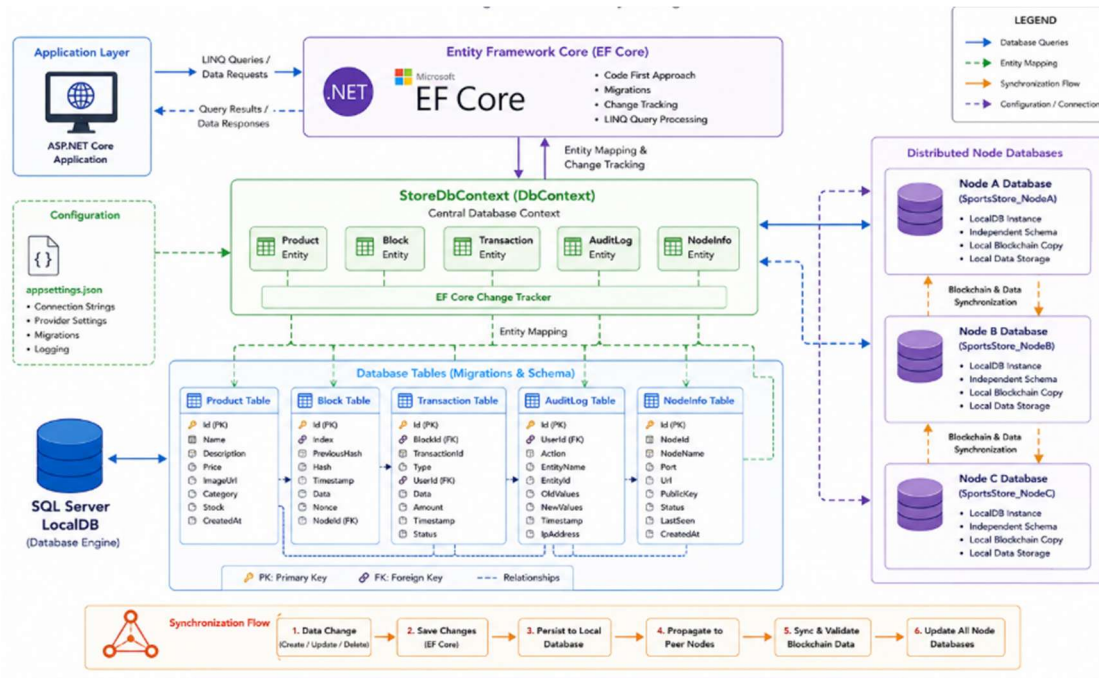


Figure 8 : Database environment architecture diagram

### 3-7. Blockchain Environment Configuration

The blockchain environment represents one of the most important components of the proposed system. The blockchain subsystem was implemented to provide secure distributed transaction validation, blockchain synchronization, and decentralized communication between nodes.

The blockchain environment was designed to simulate realistic blockchain network behavior within a local distributed environment.

The blockchain subsystem includes several dedicated services including:

- BlockchainService
- NodeService
- NodeCommunicationService
- KeyGeneratorService

These services collectively manage:

- Blockchain creation
- Block validation
- Transaction propagation
- Distributed synchronization
- Cryptographic operations
- Peer-to-peer communication

The blockchain implementation stores transaction records inside linked blockchain blocks where each block maintains:

- Transaction information
- Hash values
- Previous block references
- Validation metadata

This structure improves:

- Data integrity
- Tamper resistance
- Blockchain consistency
- Distributed trust

The blockchain environment additionally supports:

- Multi-node synchronization
- Distributed validation
- Blockchain replication
- Node-to-node communication

Each blockchain node maintains:

- Independent blockchain copy
- Local transaction history
- Distributed synchronization configuration

The blockchain environment therefore simulates decentralized ledger environments used in modern blockchain systems.

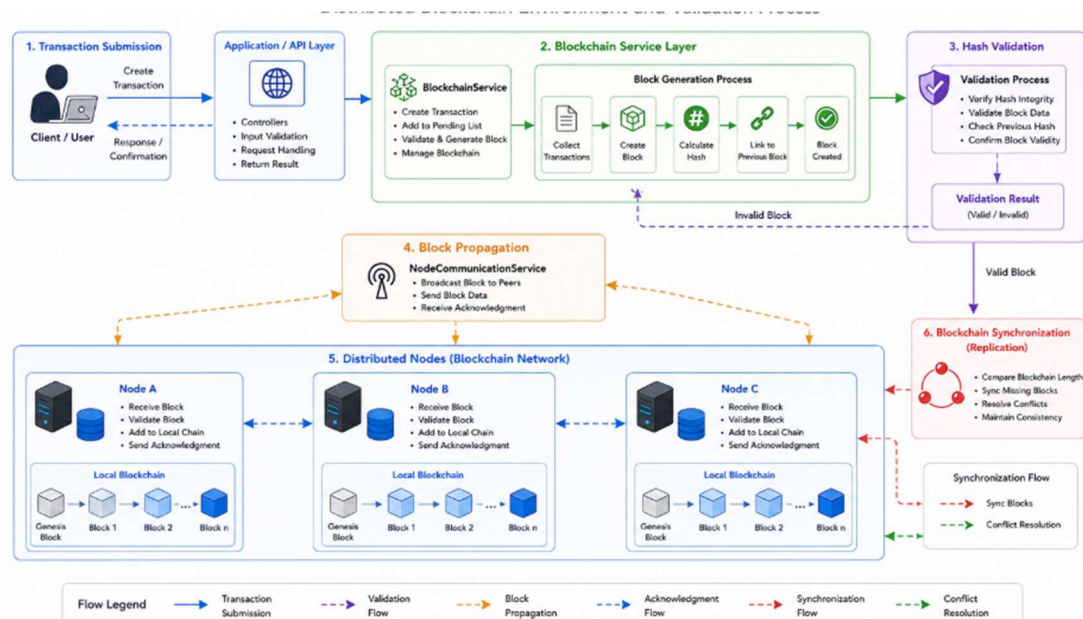


Figure 9 : Blockchain architecture overview diagram

### 3-8. Distributed Node Setup

The proposed system was configured as a distributed blockchain network composed of multiple independent nodes communicating through peer-to-peer synchronization mechanisms.

The distributed environment includes:

- Node-A
- Node-B
- Node-C

Each node operates independently using:

- Separate application configuration
- Independent communication port
- Dedicated database instance
- Local blockchain copy

The nodes communicate using REST-based communication protocols implemented through:

- NodeCommunicationService

The node configuration is managed using dedicated configuration files:

- appsettings.NodeA.json
- appsettings.NodeB.json
- appsettings.NodeC.json

Each node stores:

- Node identifier
- Peer node addresses
- Database connection strings
- Communication settings

Examples of node communication ports include:

- http://localhost:5000
- http://localhost:5001
- http://localhost:5002

The peer-to-peer communication architecture allows:

- Blockchain synchronization
- Transaction propagation
- Distributed validation

- Decentralized communication

The distributed environment was designed to emulate real decentralized blockchain systems where nodes cooperate to maintain blockchain consistency.

This architecture improves:

- Fault tolerance
- Distributed trust
- Synchronization reliability
- Blockchain scalability

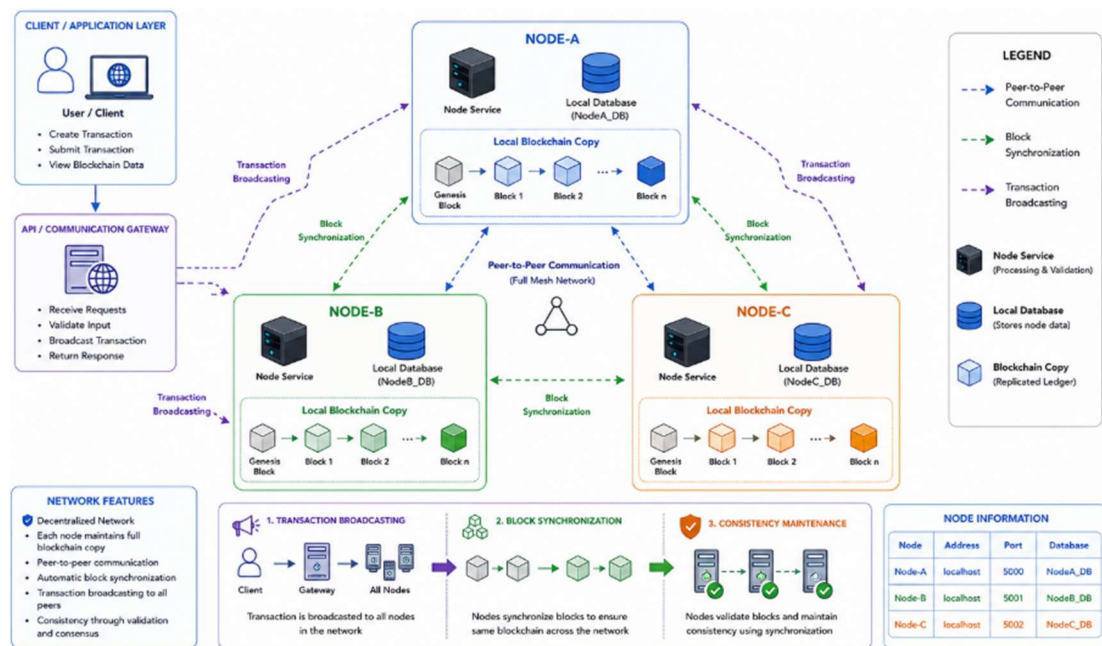


Figure 10 : Distributed blockchain network infographic

### 3-9. API and Communication Configuration

The proposed system implements RESTful APIs to support communication between frontend interfaces, backend services, and distributed blockchain nodes.

The API environment was implemented using ASP.NET Core Web APIs and supports:

- Request processing
- Blockchain communication
- Distributed synchronization
- Data exchange
- Validation requests

The APIs are documented using Swagger to simplify:

- API testing
- Endpoint monitoring

- Backend debugging
- Request validation

Swagger was configured using:

- Swashbuckle.AspNetCore
- Microsoft.AspNetCore.OpenApi

The launch environment automatically opens the Swagger interface during development startup.

The communication layer additionally supports:

- HTTP-based communication
- JSON serialization
- Distributed messaging
- Peer synchronization

The APIs play a major role in:

- Blockchain propagation
- Node communication
- Frontend-backend interaction
- Validation processing

The communication environment therefore acts as the central integration layer connecting all system components

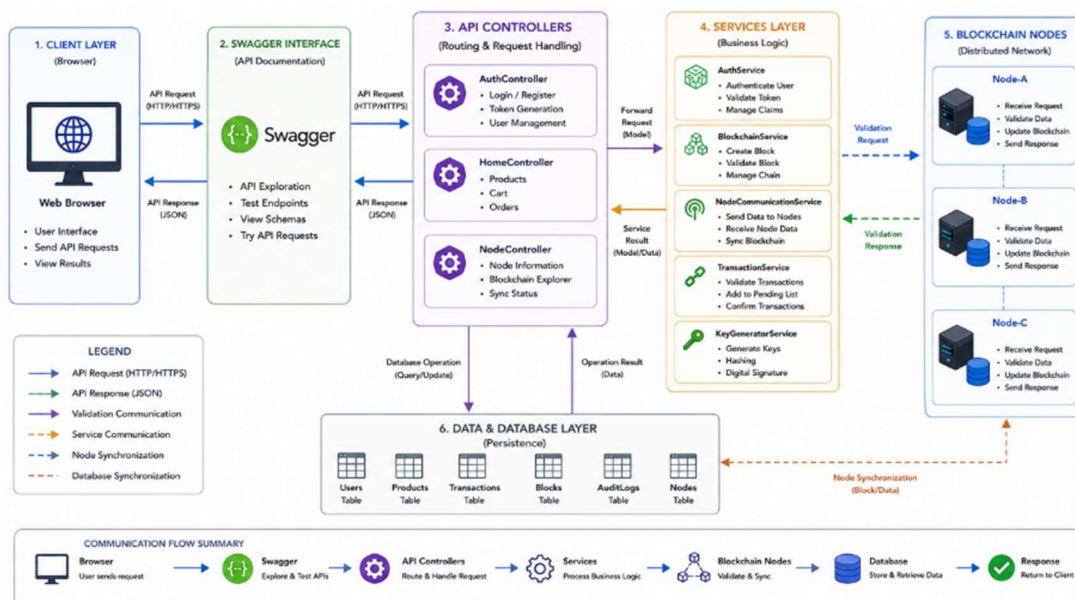


Figure 11 : REST API communication flow diagram

### 3-10. Security and Validation Environment

Security represents one of the core objectives of the proposed blockchain-based platform. The development environment integrates several security-oriented mechanisms to ensure secure communication, transaction integrity, and blockchain consistency.

The security environment includes:

- Blockchain validation
- Cryptographic operations
- Distributed verification
- Audit logging
- Request validation
- Layer isolation

The project implements:

- KeyGeneratorService
- AuditLog
- Blockchain verification mechanisms
- Multi-layer validation

Security validation occurs across several system layers including:

- Controller validation
- Service validation
- Blockchain validation
- Node verification

The blockchain subsystem improves security through:

- Immutable transaction storage
- Distributed trust
- Tamper resistance
- Transaction traceability

The audit logging subsystem records:

- Authentication operations
- Blockchain transactions
- User activities
- Validation events

The implementation of distributed validation significantly reduces the risks associated with centralized transaction manipulation.

The security environment therefore combines:

- Traditional web security
- Blockchain integrity

- Distributed trust mechanisms
- Cryptographic protection

to create a secure decentralized transaction environment.

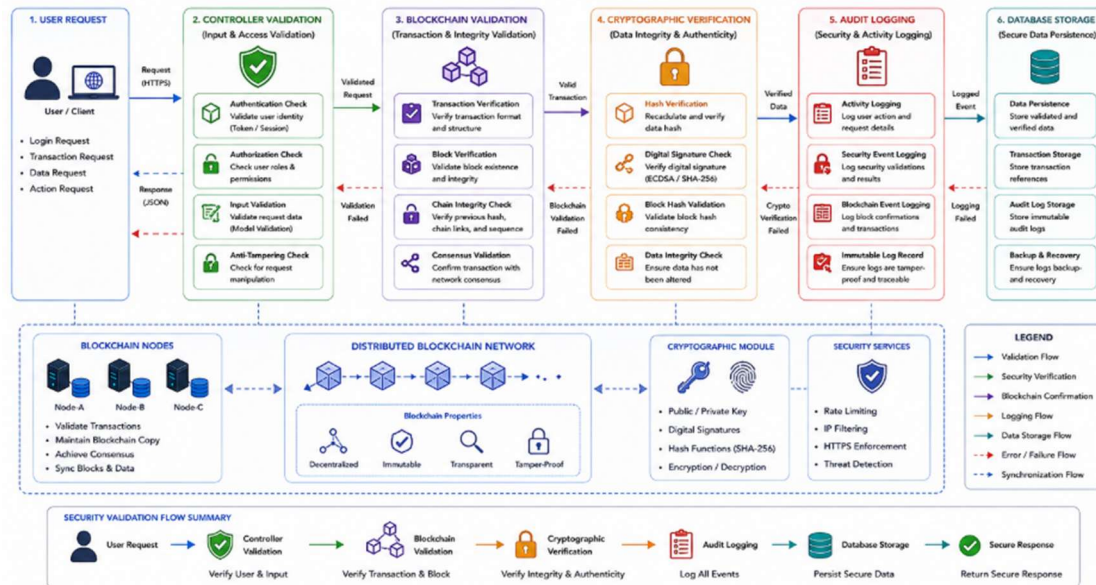


Figure 12 : Security validation workflow diagram

### 3-11. Frontend Environment Configuration

The frontend environment was implemented using Razor Pages and MVC Views to provide dynamic user interaction and reusable interface components.

The frontend subsystem includes:

- Shared layouts
- Navigation components
- Blockchain explorer pages
- Shopping cart interfaces
- Product summary views

Examples include:

- \_Layout.cshtml
- BlockchainExplorer.cshtml
- Cart.cshtml
- ProductSummary.cshtml

The frontend environment was designed to provide:

- User-friendly interaction
- Dynamic rendering
- Reusable UI components
- Blockchain monitoring interfaces

The project also implements reusable frontend components such as:

- NavigationMenuViewComponent
- PageLinkTagHelper

These components improve:

- Navigation management
- Pagination handling
- Interface consistency
- Reusability

The blockchain explorer interface allows users to:

- Inspect blockchain records
- Monitor transactions
- Visualize distributed blockchain data

The frontend layer communicates with backend services exclusively through controllers and APIs, improving system security and preventing direct database access from the client side.

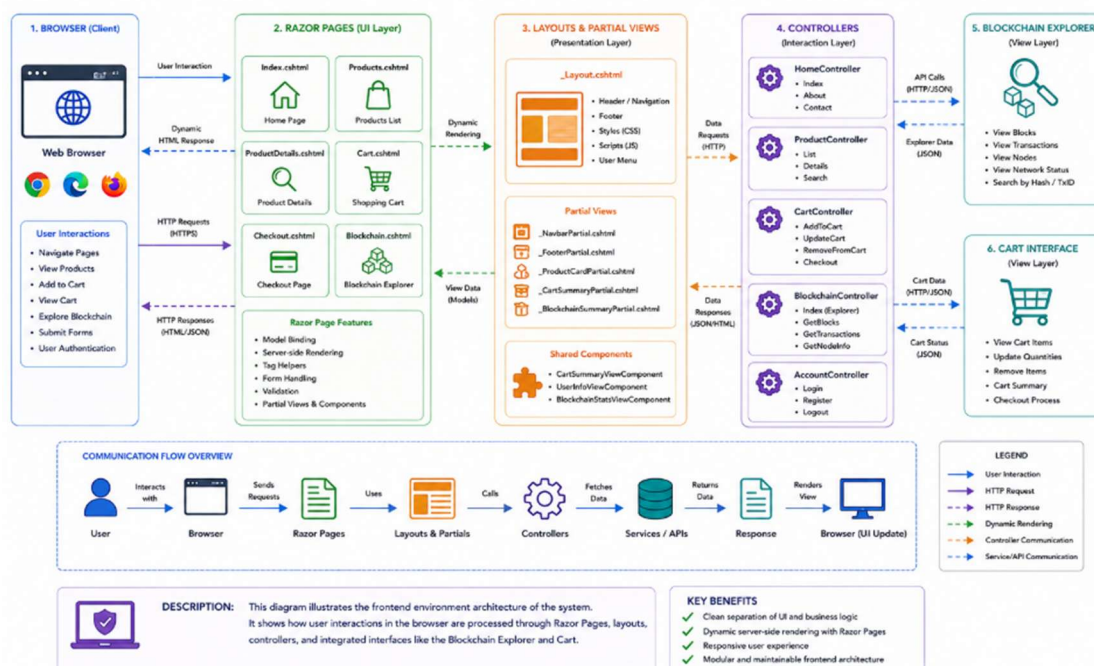


Figure 13 : Frontend architecture diagram overview

### 3-12. System Workflow

The proposed system follows a distributed blockchain workflow where transactions pass through several validation and synchronization stages before being permanently stored within the blockchain environment.

The workflow begins when the user submits a request through the frontend interface. The request is received by the controller layer where preliminary validation is performed.

After validation, the request is forwarded to the service layer where:

- Business logic is processed
- Blockchain validation is initiated
- Transactions are prepared

The BlockchainService then:

- Validates transaction integrity
- Creates blockchain blocks
- Generates blockchain records

After successful validation, the blockchain information is propagated to neighboring nodes through:

- NodeCommunicationService

The distributed nodes synchronize blockchain updates and maintain consistent blockchain copies across the decentralized environment.

Finally:

- The transaction is stored inside the local database
- Audit logs are generated
- Blockchain explorer interfaces are updated

This workflow ensures:

- Distributed trust
- Secure transaction validation
- Blockchain consistency
- Tamper resistance
- Decentralized synchronization

The complete system workflow therefore integrates:

- MVC architecture
- Blockchain validation
- Distributed communication
- Database persistence
- Security monitoring

within a unified decentralized environment.

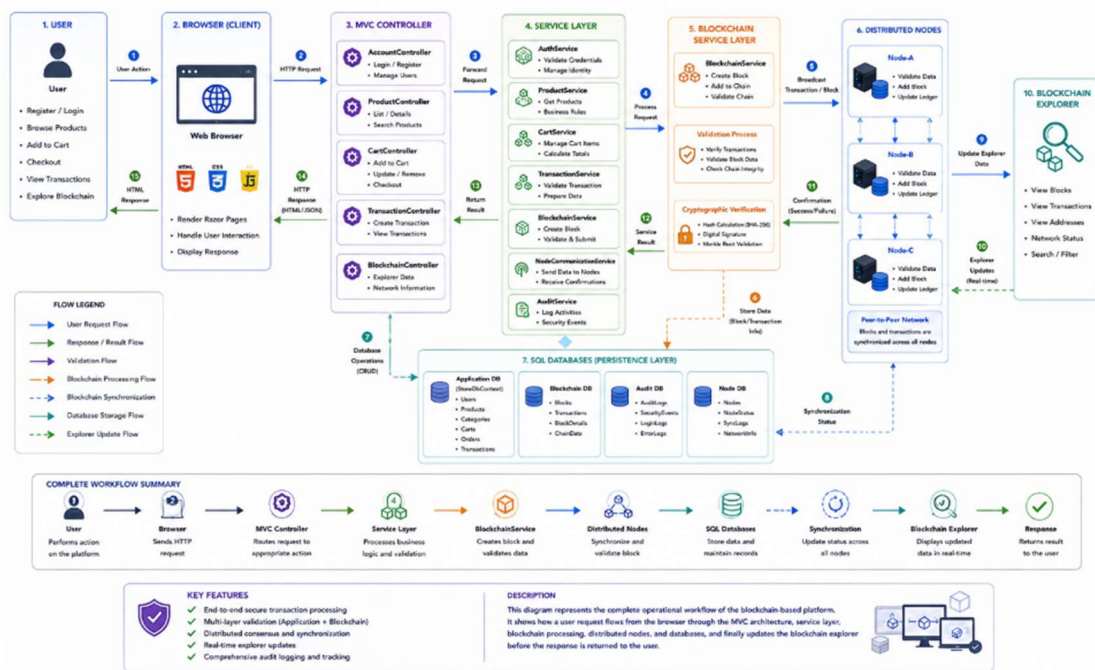


Figure 14 : Complete blockchain system workflow diagram

### 3-13. Conclusion

his chapter presented the complete development environment and implementation configuration of the proposed blockchain-based platform. The chapter discussed the hardware and software requirements, backend and frontend environments, database setup, blockchain architecture, distributed node communication, API configuration, and security mechanisms.

he implemented environment successfully simulated a decentralized blockchain network capable of supporting secure transaction validation, distributed synchronization, and blockchain-based data integrity verification. The integration of ASP.NET Core MVC, Entity Framework Core, SQL Server LocalDB, and distributed blockchain services provided a scalable and secure implementation environment suitable for the proposed system objectives.

## **Chapter 4: The Proposed Solution**

## 4-1. Introduction

The proposed solution was developed to address the security, integrity, and trust limitations commonly found in traditional centralized e-commerce systems. Conventional web-based transaction platforms typically depend on centralized databases and single-server architectures, which create several security risks including data tampering, unauthorized transaction modification, single points of failure, and limited transaction traceability.

To overcome these limitations, the proposed project introduces a blockchain-based distributed e-commerce security platform that combines modern web technologies with decentralized blockchain mechanisms. The system was designed to provide secure transaction processing, distributed validation, blockchain synchronization, cryptographic verification, and audit monitoring within a multi-node distributed environment.

The proposed platform integrates:

- Distributed blockchain architecture
- Secure transaction validation
- Multi-node synchronization
- Cryptographic verification mechanisms
- Blockchain replication
- RESTful API communication
- MVC-based web architecture
- Distributed database environments
- Audit logging systems
- Blockchain explorer interfaces

Unlike traditional centralized systems where transaction validation occurs on a single server, the proposed platform distributes validation operations across multiple blockchain nodes. Each node maintains an independent blockchain copy and participates in transaction verification and synchronization processes.

The proposed solution was implemented using ASP.NET Core MVC together with Entity Framework Core, SQL Server LocalDB, blockchain services, and distributed communication protocols. The environment simulates a decentralized blockchain network where multiple nodes communicate securely using peer-to-peer synchronization.

The implementation architecture focuses on:

- Data integrity
- Decentralization
- Secure synchronization
- Transaction immutability
- Distributed trust
- Multi-layer validation
- Fault tolerance

This chapter explains the proposed system architecture, implementation methodology, blockchain workflow, communication mechanisms, validation processes, database architecture, security protocols, and distributed synchronization environment used throughout the project.

## 4-2. Proposed System Overview

The proposed system is a blockchain-based e-commerce security platform designed to provide secure transaction processing and distributed validation using blockchain technology. The system combines traditional web application architecture with decentralized blockchain mechanisms in order to improve security, integrity, and transparency.

The platform operates using a distributed multi-node architecture where blockchain operations are shared between several independent nodes. Each node maintains:

- Independent blockchain copy
- Local database instance
- Distributed synchronization settings
- Peer node communication configuration
- Blockchain validation capabilities

The system architecture consists of several major components including:

**Table 8 : Main System Components and Their Responsibilities**

| Component                | Responsibility                            |
|--------------------------|-------------------------------------------|
| MVC Controllers          | Request processing and routing            |
| Service Layer            | Business logic and validation             |
| Blockchain Layer         | Transaction validation and block creation |
| Node Communication Layer | Distributed synchronization               |
| Database Layer           | Persistent blockchain storage             |
| Frontend Layer           | User interaction and visualization        |
| Audit Layer              | Security monitoring and logging           |

The proposed solution enables users to:

- Browse products
- Manage shopping cart operations
- Submit transactions
- Monitor blockchain records
- View transaction history
- Inspect blockchain synchronization

The system additionally provides administrative and blockchain monitoring functionality through a dedicated blockchain explorer interface.

The overall architecture was designed according to:

- Separation of Concerns (SoC)
- MVC architectural principles
- Service-oriented architecture
- Distributed blockchain design
- Multi-layer validation architecture

These architectural decisions improved:

- Scalability
- Maintainability
- Security
- Modularity
- Distributed fault tolerance

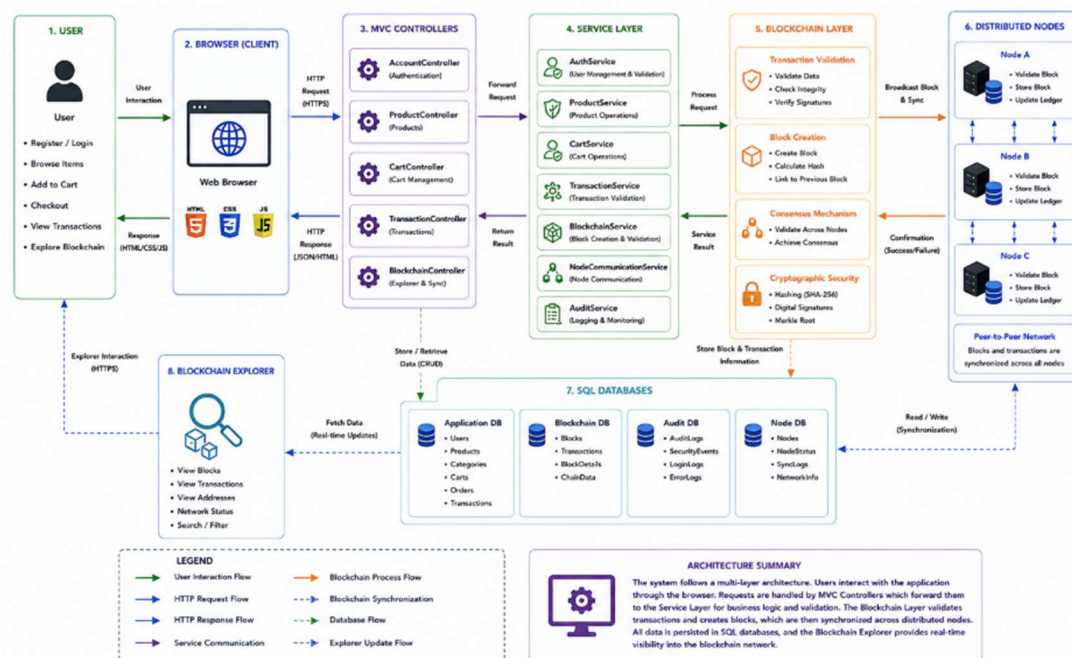


Figure 15 : System Overview Architecture Diagram

### 4-3. Proposed System Architecture

The proposed platform follows a layered distributed architecture designed to support secure blockchain processing, decentralized synchronization, and scalable web application functionality.

The architecture separates the system into several independent layers where each layer performs a specialized responsibility.

The implemented layers include:

- Presentation Layer
- Controller Layer
- Service Layer
- Repository Layer
- Blockchain Layer
- Database Layer
- Distributed Node Layer
- Security and Validation Layer

This layered architecture reduces coupling between system components and simplifies future system maintenance and scalability.

The architecture additionally supports:

- Distributed blockchain synchronization
- Secure transaction propagation
- RESTful API communication
- Blockchain replication
- Cryptographic validation
- Audit monitoring

The implementation environment combines:

- ASP.NET Core MVC
- Entity Framework Core
- SQL Server LocalDB
- Razor Pages
- Blockchain services
- Peer-to-peer node communication

The layered architecture also improves system security because validation occurs at multiple independent layers including:

- Controller validation
- Service validation
- Blockchain validation
- Cryptographic verification
- Distributed node confirmation

This multi-layer validation model significantly reduces the risk of invalid or malicious transactions entering the blockchain environment.

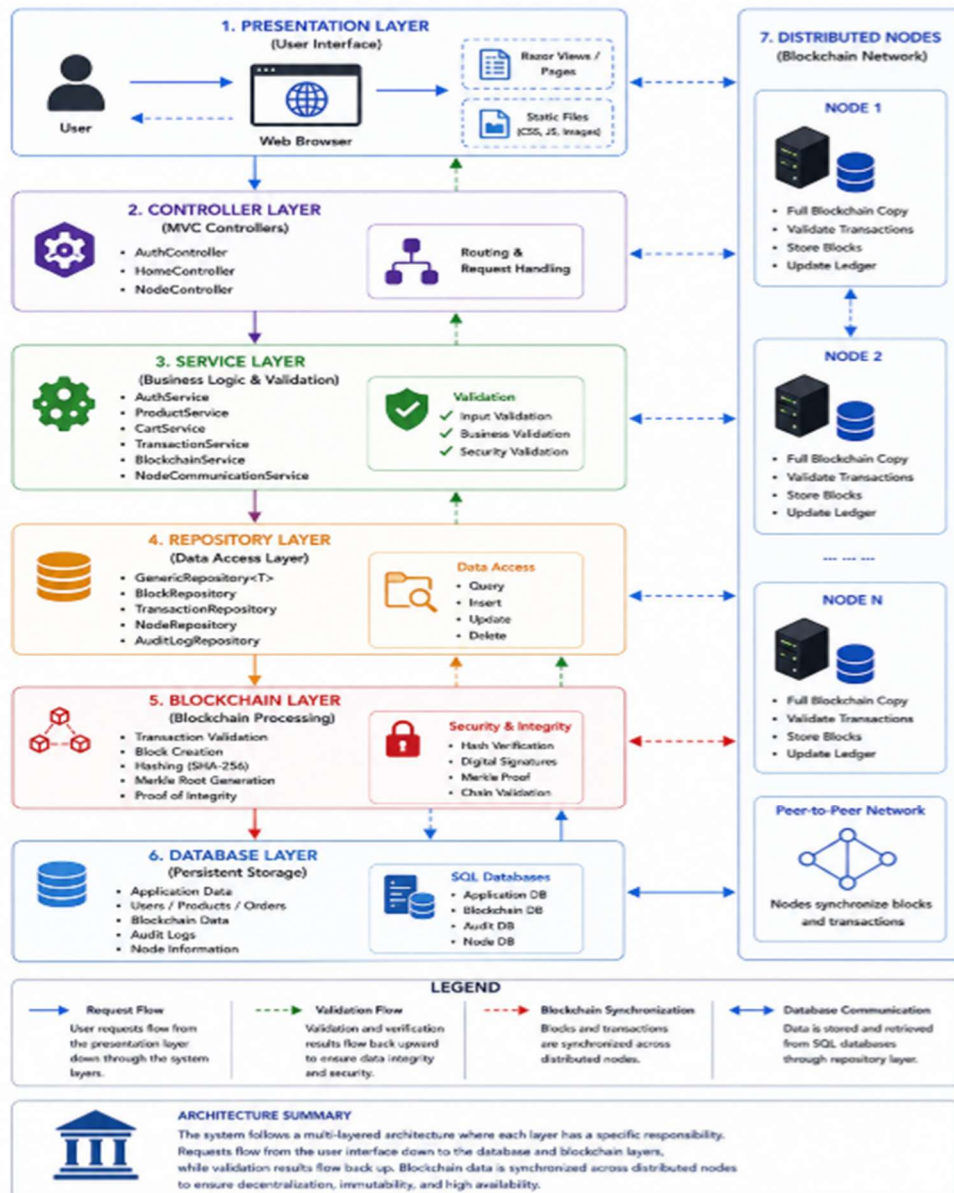


Figure 16 : Layered System Architecture Diagram

#### 4-4. MVC-Based Application Design

The proposed solution implements the MVC (Model-View-Controller) architectural pattern to organize system components and improve separation between frontend rendering, business logic, and data processing.

The MVC architecture was selected because it provides:

- Modular application structure

- Clear separation of responsibilities
- Improved maintainability
- Reusable frontend components
- Centralized request processing
- Simplified testing and debugging

The MVC architecture consists of three primary components:

- Models
- Views
- Controllers

#### **4-4.1. Model Layer**

The Model layer represents the core business entities and blockchain-related data structures used throughout the system.

The project contains several models including:

- Product
- Cart
- CartTransaction
- Block
- NodeInfo
- AuditLog
- NodeRecord

The model layer stores:

- Product information
- Blockchain records
- Transaction histories
- Audit logs
- Distributed node information

The model architecture additionally supports:

- Entity validation
- Relationship management
- Blockchain consistency
- Distributed synchronization

#### **4-4.2. View Layer**

The View layer was implemented using Razor Pages and MVC Views in order to provide dynamic frontend rendering and reusable user interface components.

The frontend includes:

- Shopping cart interfaces
- Blockchain explorer pages
- Product pages
- Navigation menus
- Shared layouts

- Partial views

The frontend architecture improves:

- User interaction
- Dynamic rendering
- Navigation consistency
- Reusability

#### 4-4.3. Controller Layer

The Controller layer handles incoming HTTP requests and coordinates communication between frontend interfaces, backend services, blockchain components, and database layers.

The project includes several controllers including:

- AuthController
- HomeController
- NodeController

The controllers are responsible for:

- Request routing
- Validation
- Service communication
- Blockchain interaction
- API response handling

The controller layer also acts as the first validation checkpoint before requests reach the blockchain environment.

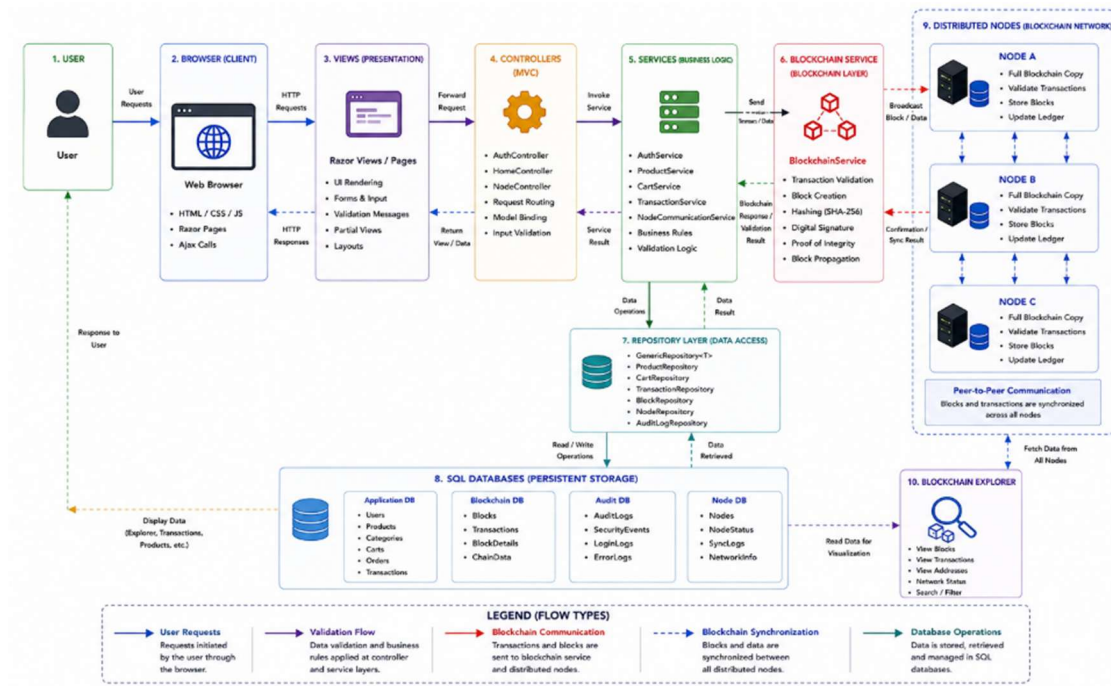


Figure 17 : MVC Security-Oriented Architecture Diagram

## 4-5. Blockchain-Based Solution Design

Blockchain technology represents the core component of the proposed solution. The blockchain subsystem was designed to provide secure transaction validation, decentralized synchronization, immutable storage, and distributed trust management.

Unlike centralized transaction systems where all records are controlled by a single server, the proposed blockchain architecture distributes transaction validation across multiple independent nodes.

The blockchain subsystem includes:

- BlockchainService
- NodeService
- NodeCommunicationService
- KeyGeneratorService

The blockchain environment supports:

- Block creation
- Transaction validation
- Blockchain synchronization
- Hash verification
- Distributed replication
- Peer-to-peer communication

Each blockchain block stores:

- Transaction data
- Previous block hash
- Current block hash
- Timestamp information
- Validation metadata

This blockchain structure improves:

- Data integrity
- Tamper resistance
- Traceability
- Distributed trust
- Transaction immutability

The blockchain workflow operates through several stages:

1. Transaction submission
2. Request validation
3. Blockchain validation
4. Hash generation
5. Block creation

6. Block propagation
7. Node synchronization
8. Database persistence

The blockchain subsystem additionally supports:

- Distributed consensus simulation
- Blockchain replication
- Multi-node synchronization
- Fault-tolerant validation

The implementation therefore simulates realistic decentralized blockchain environments used in secure distributed transaction systems.

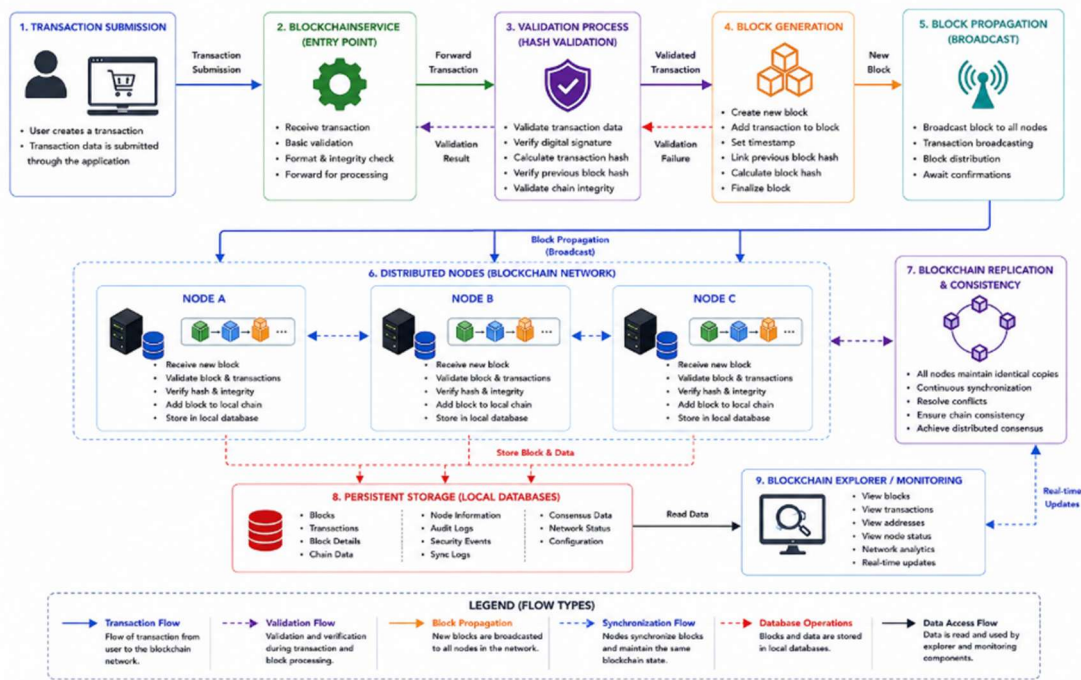


Figure 18 : Blockchain Architecture Diagram

#### 4-6. Distributed Node Communication

The proposed solution implements a distributed blockchain network composed of multiple independent nodes communicating through peer-to-peer synchronization.

The distributed environment consists of:

- Node-A
- Node-B
- Node-C

Each node operates independently with:

- Separate database instance
- Independent blockchain copy
- Dedicated communication endpoint
- Node-specific configuration

The distributed node architecture was designed to simulate decentralized blockchain behavior where nodes cooperate to maintain synchronized blockchain copies.

Node communication is handled using:

- NodeCommunicationService
- REST APIs
- HTTP-based communication

Each node stores peer node addresses inside dedicated configuration files including:

- appsettings.NodeA.json
- appsettings.NodeB.json
- appsettings.NodeC.json

The communication layer supports:

- Transaction broadcasting
- Block propagation
- Blockchain synchronization
- Distributed validation
- Peer discovery

The distributed environment significantly improves:

- Decentralization
- Fault tolerance
- Blockchain consistency
- Distributed trust
- Synchronization reliability

The system therefore emulates real blockchain network behavior where blockchain updates are propagated and synchronized across multiple distributed participants.

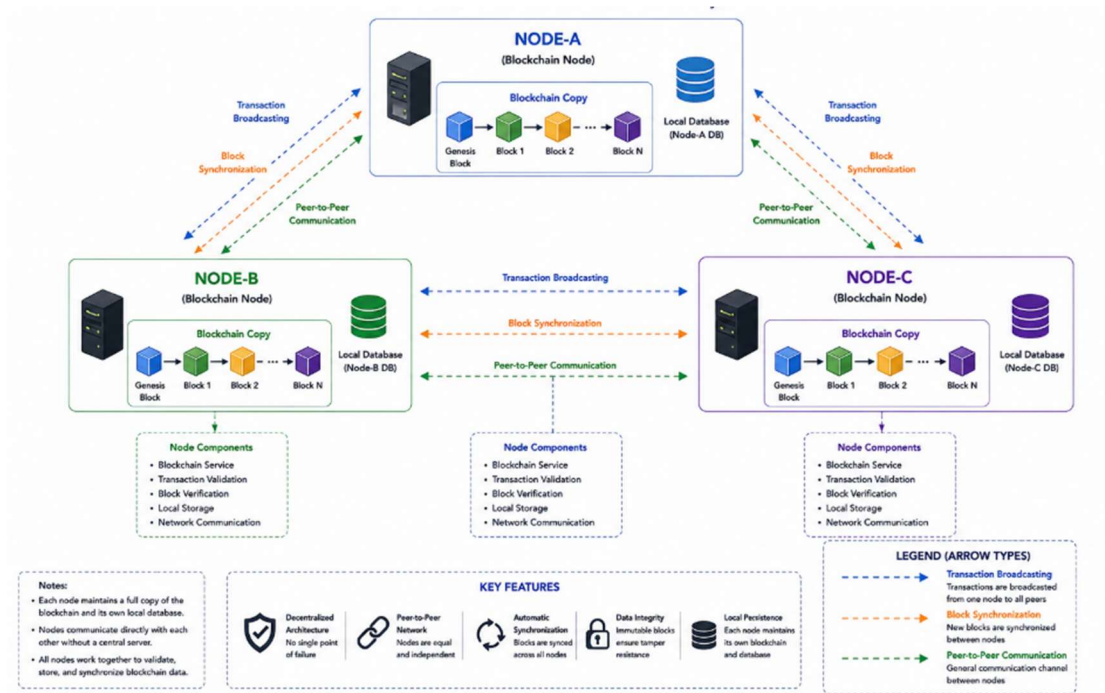


Figure 19 : Distributed Blockchain Network Diagram

## 4-7. Database Design and Storage Architecture

The proposed platform implements a distributed relational database architecture using SQL Server LocalDB together with Entity Framework Core.

The database environment stores:

- Blockchain records
- Product information
- Transaction histories
- Cart operations
- Audit logs
- Node metadata

The project uses:

- StoreDbContext
- Entity Framework Core migrations
- Repository Pattern
- Distributed database instances

Each blockchain node operates using an independent database instance including:

- SportsStore\_NodeA
- SportsStore\_NodeB
- SportsStore\_NodeC

This distributed database architecture simulates decentralized blockchain environments where each node stores its own blockchain records and transaction history.

The system includes several database tables including:

- Product Table
- Block Table
- Transaction Table
- AuditLog Table
- NodeInfo Table

The database architecture additionally supports:

- Entity mapping
- Distributed synchronization
- Blockchain persistence
- Audit monitoring
- Transaction storage

Entity Framework Core was used to simplify:

- Database communication
- Query execution
- Migration management
- Relationship handling

The Repository Pattern was implemented to isolate business logic from direct database interaction.

This architecture improves:

- Maintainability
- Data abstraction
- Scalability
- Code organization

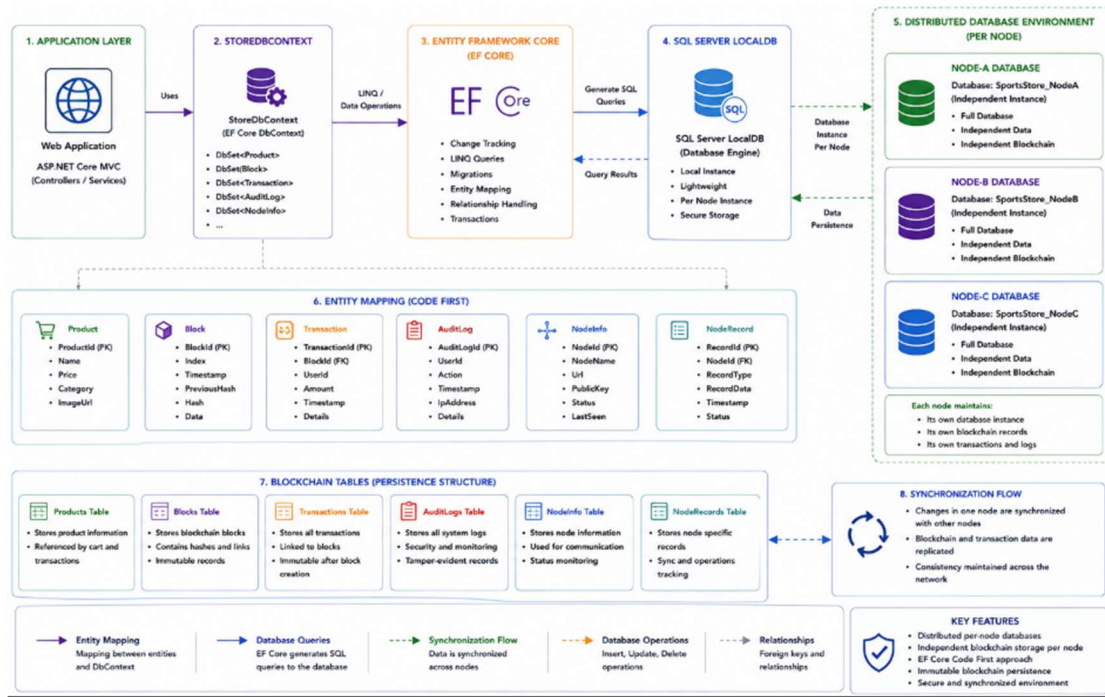


Figure 20 : Database Environment Architecture Diagram

## 4-8. Security and Validation Mechanisms

Security represents one of the primary objectives of the proposed blockchain-based platform. The system integrates several validation and cryptographic mechanisms to ensure secure transaction processing and blockchain integrity.

The security architecture includes:

- Controller validation
- Blockchain validation
- Cryptographic verification
- Audit logging
- Distributed node confirmation
- Blockchain synchronization checks

The validation workflow begins when the user submits a request through the frontend interface. The request then passes through several validation layers including:

1. Input validation
2. Authentication verification
3. Blockchain validation
4. Cryptographic verification
5. Distributed confirmation
6. Database persistence

The system uses:

- KeyGeneratorService
- BlockchainService
- AuditLog
- Distributed synchronization validation

The cryptographic subsystem supports:

- Hash generation
- Secure validation
- Blockchain integrity verification
- Transaction authenticity

The audit logging subsystem records:

- User activities
- Blockchain operations
- Validation events
- Transaction confirmations

This security-oriented architecture improves:

- Data integrity
- Tamper resistance
- Accountability
- Distributed trust
- Traceability

The combination of blockchain validation and cryptographic verification significantly reduces the risk of unauthorized transaction modification.

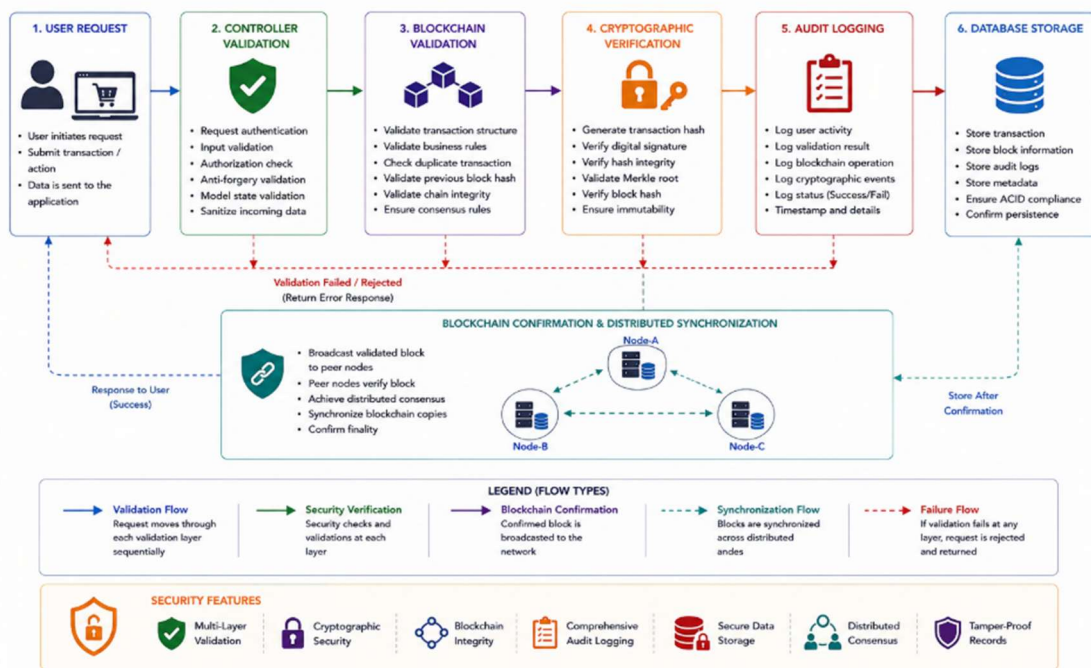


Figure 21 : Security Validation Workflow Diagram

## 4-9. REST API and Communication Protocols

The proposed platform implements RESTful APIs to support communication between frontend interfaces, backend services, and distributed blockchain nodes.

The API architecture supports:

- Request processing
- Blockchain synchronization
- Node communication
- Transaction validation
- Distributed messaging

The APIs were implemented using ASP.NET Core Web APIs together with Swagger for interactive testing and endpoint documentation.

The communication architecture additionally supports:

- HTTP communication
- JSON serialization
- RESTful routing
- Peer-to-peer synchronization

Swagger was integrated to simplify:

- API testing
- Request validation
- Backend debugging
- Endpoint monitoring

The API environment acts as the integration layer connecting:

- Frontend interfaces
- Controllers
- Blockchain services
- Distributed nodes
- Database systems

The communication architecture therefore enables secure distributed request processing across the blockchain environment.

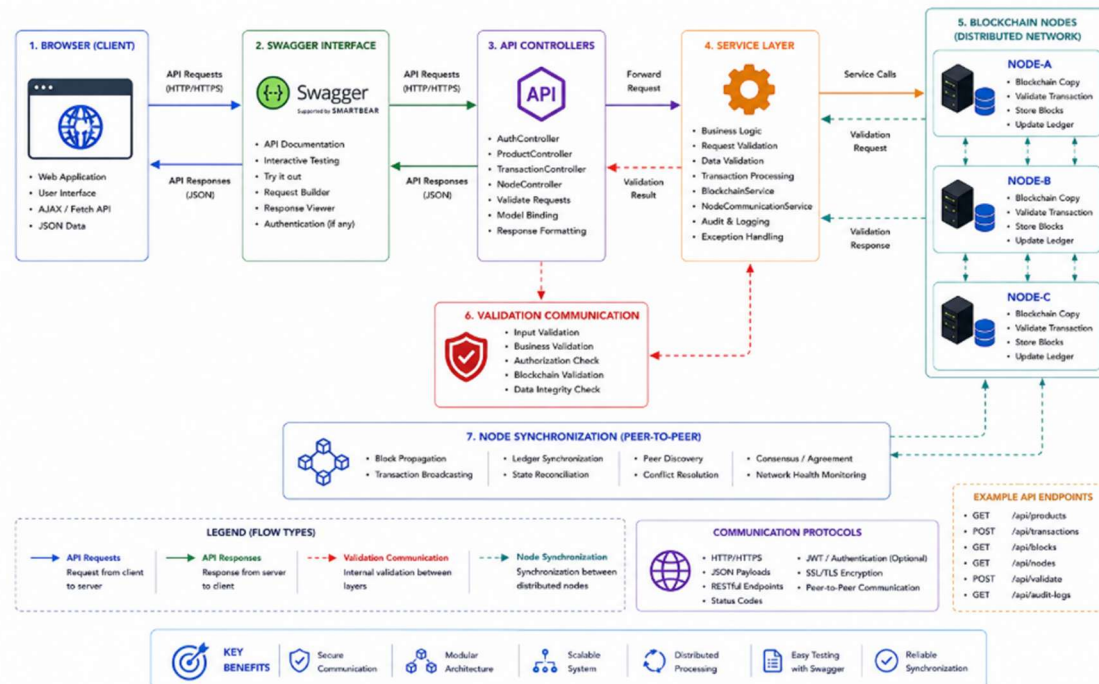


Figure 22 : REST API Communication Diagram

## 4-10. Frontend Design and User Interaction

The frontend environment was implemented using Razor Pages and MVC Views to provide dynamic rendering and interactive user interfaces.

The frontend subsystem includes:

- Product pages
- Shopping cart interfaces
- Blockchain explorer pages
- Shared layouts
- Navigation menus
- Partial views

The frontend architecture was designed to improve:

- User experience

- Dynamic rendering
- Interface modularity
- Reusability

The blockchain explorer interface allows users to:

- View blockchain records
- Monitor transactions
- Inspect blockchain synchronization
- Explore distributed node data

The shopping cart interface enables:

- Product selection
- Cart management
- Transaction submission
- Checkout processing

The frontend communicates with backend services through:

- MVC Controllers
- REST APIs
- Service Layer

The separation between frontend rendering and backend processing improves:

- Security
- Maintainability
- Scalability
- User interaction management

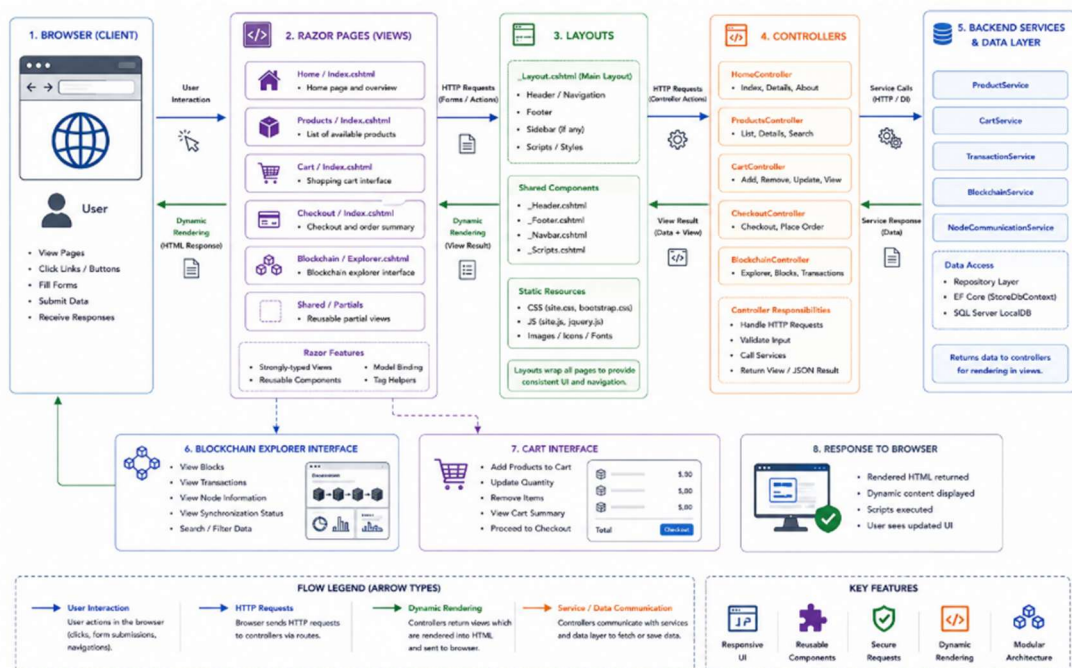


Figure 23 : Frontend Environment Architecture Diagram

## 4-11. Complete System Workflow

The proposed system follows a distributed blockchain operational workflow where transactions pass through multiple processing and validation stages before being permanently stored within the blockchain environment.

The workflow begins when the user interacts with the frontend environment through the browser interface.

The operational flow proceeds as follows:

1. User submits a request
2. Browser sends HTTP request
3. MVC Controller processes request
4. Service Layer executes business logic
5. BlockchainService validates transaction
6. Block is generated and verified
7. Distributed nodes synchronize blockchain updates
8. Data is stored in SQL databases
9. Blockchain explorer is updated
10. Response is returned to the user

The workflow integrates:

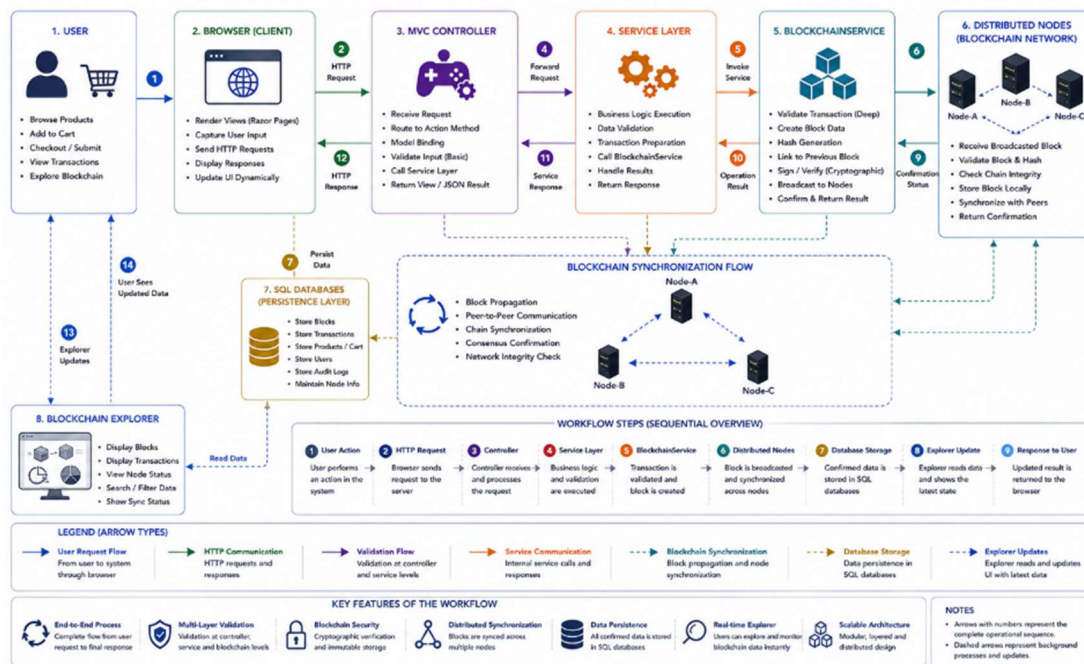
- MVC architecture
- Blockchain validation
- Distributed synchronization
- Database persistence
- Cryptographic verification
- Audit logging

The synchronization layer ensures that all distributed nodes maintain consistent blockchain copies.

The workflow additionally supports:

- Distributed trust
- Transaction immutability
- Secure propagation
- Multi-layer validation

This architecture therefore provides a secure decentralized transaction processing environment suitable for blockchain-based e-commerce systems.



## 4-12. Proposed Algorithms and Validation Logic

The proposed solution implements several validation and synchronization mechanisms to maintain blockchain consistency and secure transaction processing.

The validation logic follows sequential multi-layer verification including:

- Request validation
- Transaction verification
- Blockchain integrity checking
- Hash validation
- Distributed synchronization
- Database confirmation

The blockchain validation algorithm performs:

1. Receive transaction
2. Validate transaction structure
3. Generate hash value
4. Verify previous block hash
5. Create blockchain block
6. Broadcast block to peer nodes
7. Synchronize blockchain copies

## 8. Persist blockchain data

The synchronization process ensures:

- Blockchain consistency
- Distributed replication
- Fault-tolerant operation
- Decentralized verification

The project additionally implements:

- Repository Pattern
- Dependency Injection
- Distributed configuration loading
- Session-based transaction management

These implementation mechanisms improve:

- Modularity
- Maintainability
- Scalability
- Security

## 4-13. Conclusion

This chapter presented the proposed solution and implementation methodology of the blockchain-based e-commerce security platform. The chapter explained the overall system architecture, blockchain environment, distributed node communication, database architecture, frontend and backend design, REST API communication, security mechanisms, and complete operational workflow.

The proposed solution successfully integrates distributed blockchain validation with modern web application technologies in order to provide secure transaction processing, blockchain synchronization, distributed trust management, and decentralized data integrity verification.

The integration of ASP.NET Core MVC, Entity Framework Core, SQL Server LocalDB, blockchain services, distributed nodes, and cryptographic validation mechanisms created a scalable and secure distributed blockchain platform suitable for modern secure transaction environments.

## **Chapter 5: Comparisons and Results**

## **4-1. Introduction**

This chapter presents the practical results obtained after implementing and testing the proposed blockchain-based e-commerce security platform. The purpose of this chapter is to evaluate the effectiveness of the proposed system in terms of blockchain validation, distributed synchronization, API communication, database persistence, security verification, and overall system functionality.

The implemented platform was tested within a distributed blockchain environment consisting of multiple interconnected nodes operating simultaneously through peer-to-peer synchronization mechanisms. The evaluation process focused on verifying the operational stability of the platform and ensuring that all blockchain operations were executed correctly across the distributed environment.

The testing and evaluation phase included:

- Blockchain transaction validation
- Distributed node synchronization
- REST API communication testing
- Blockchain replication
- Database persistence verification
- Security and validation testing
- Blockchain explorer monitoring
- System workflow execution

The results presented in this chapter demonstrate the ability of the proposed system to maintain blockchain integrity, distributed trust, transaction immutability, and synchronized blockchain copies across multiple nodes.

Additionally, this chapter compares the proposed platform with traditional centralized systems and previous blockchain-based studies in order to highlight the improvements and additional features introduced by the proposed solution.

## **4-2. System Execution Results**

The proposed blockchain-based platform was successfully executed within a distributed development environment consisting of multiple blockchain nodes operating simultaneously.

```

[STARTUP] SeedData disabled for distributed runtime stability
Application is fully initialized and ready!
Node is listening on: http://localhost:5001

[HEARTBEAT] Service started instance=665213caeb0e48b589828255ce9e4043
[HEARTBEAT] Tick=1 instance=665213caeb0e48b589828255ce9e4043 node= bootstrapped=False clusterNodes=1
[SYNC] Pending replay start service=aca49e6c84554c288280a4f69c472214 queueCount=0
[SYNC] Pending replay end service=aca49e6c84554c288280a4f69c472214 processed=0 queueCount=0
info: Microsoft.Hosting.Lifetime[14]
    Now listening on: http://localhost:5001
info: Microsoft.Hosting.Lifetime[0]
    Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
    Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]

```

Figure 25 : Execution of Node-A in the Distributed Blockchain Environment

```

--- Node Initialization ---
Node ID (before registration):
-----
[STARTUP] SeedData disabled for distributed runtime stability
Application is fully initialized and ready!
Node is listening on: http://localhost:5002

[HEARTBEAT] Service started instance=e2a8db782a6448228614c761a8d3f570
[HEARTBEAT] Tick=1 instance=e2a8db782a6448228614c761a8d3f570 node= bootstrapped=False clusterNodes=1
[SYNC] Pending replay start service=cc74994d28504a33ade61ae2d65838f5 queueCount=0
[SYNC] Pending replay end service=cc74994d28504a33ade61ae2d65838f5 processed=0 queueCount=0
info: Microsoft.Hosting.Lifetime[14]
    Now listening on: http://localhost:5002
info: Microsoft.Hosting.Lifetime[0]
    Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
    Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]

```

Figure 26 : Execution of Node-B in the Distributed Blockchain Environment

```

--- Node Initialization ---
Node ID (before registration):
-----
[STARTUP] SeedData disabled for distributed runtime stability
Application is fully initialized and ready!
Node is listening on: http://localhost:5003

[HEARTBEAT] Service started instance=7f0e3283fa3b4d3ca137cc6ca58c2567
[HEARTBEAT] Tick=1 instance=7f0e3283fa3b4d3ca137cc6ca58c2567 node= bootstrapped=False clusterNodes=1
[SYNC] Pending replay start service=13274a884a9c4d72b230f0659aa7f373 queueCount=0
[SYNC] Pending replay end service=13274a884a9c4d72b230f0659aa7f373 processed=0 queueCount=0
info: Microsoft.Hosting.Lifetime[14]
    Now listening on: http://localhost:5003
info: Microsoft.Hosting.Lifetime[0]
    Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
    Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]

```

Figure 27 : Execution of Node-C in the Distributed Blockchain Environment

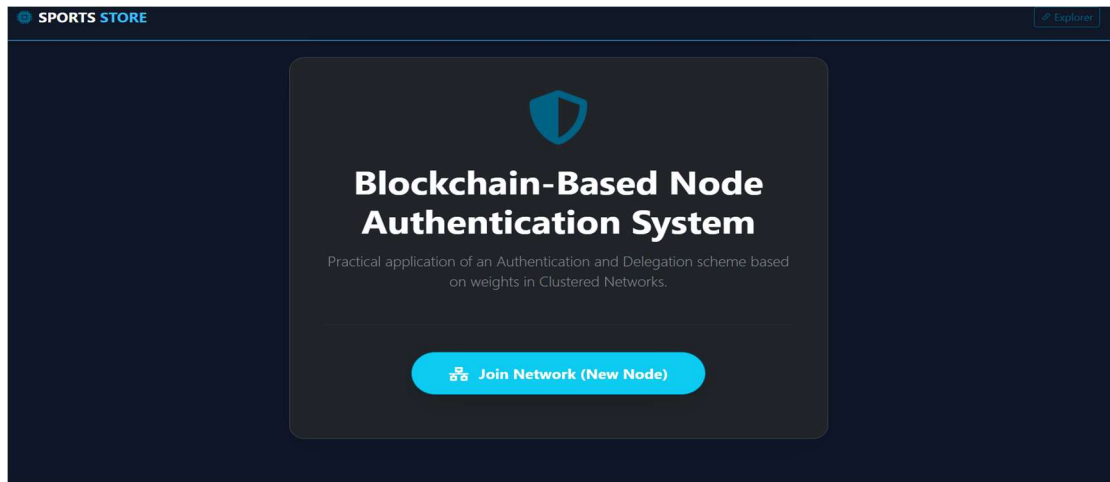


Figure 28 : Main Browser Interface of the Proposed Platform

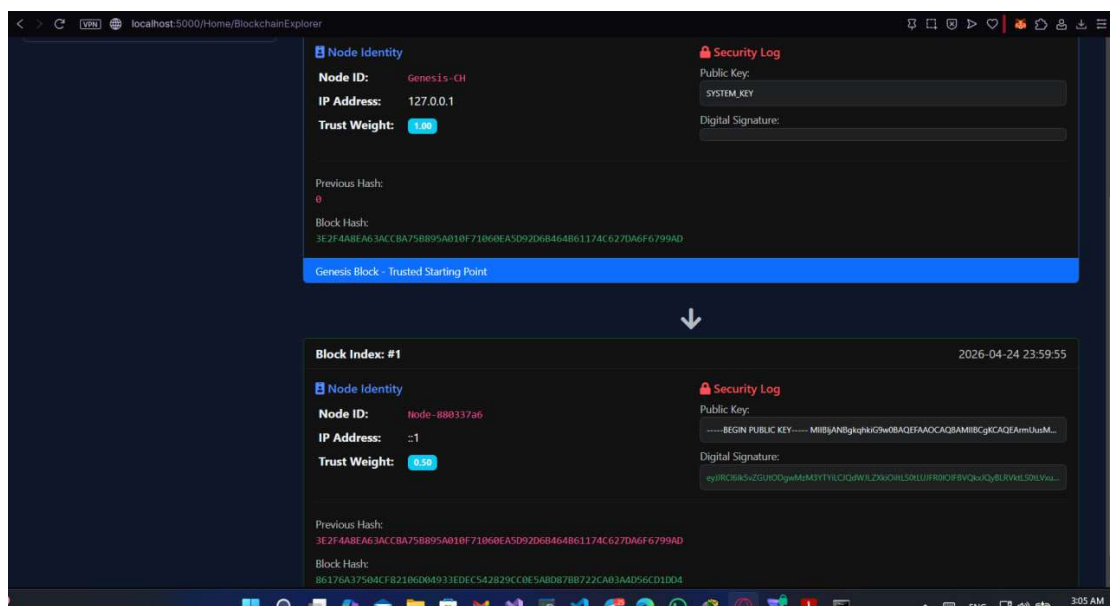


Figure 29 : Blockchain Explorer Displaying Distributed Blockchain Records

The execution environment included:

- Node-A
- Node-B
- Node-C

Each node was configured using:

- Independent database instance
- Separate blockchain copy

- Dedicated communication endpoint
- Node-specific configuration settings

The system execution results demonstrated successful:

- Application startup
- Node communication
- Blockchain synchronization
- API execution
- Frontend rendering
- Database connectivity

The platform successfully handled:

- Product browsing
- Shopping cart operations
- Transaction submissions
- Blockchain updates
- Distributed synchronization requests

The implementation environment additionally verified:

- Stable node communication
- Successful blockchain propagation
- Persistent database storage
- Real-time explorer updates

The blockchain explorer interface successfully displayed:

- Blockchain blocks
- Transaction records
- Node synchronization status
- Distributed blockchain information

The successful execution of the distributed environment confirmed that the platform was capable of operating as a decentralized blockchain-based transaction system.

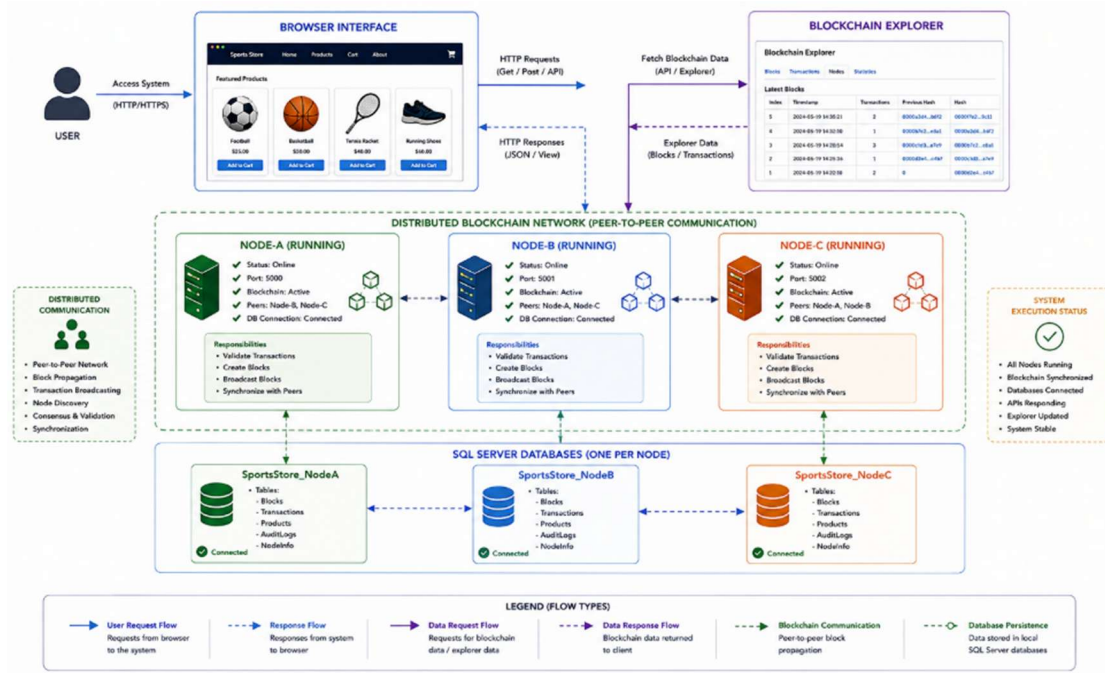


Figure 30 : System Execution Result Diagram

### 4-3. Blockchain Validation Results

The blockchain validation subsystem was tested to verify the integrity and correctness of blockchain operations performed by the proposed platform.

The validation process successfully confirmed:

- Transaction verification
- Block creation
- Hash generation
- Previous hash linkage
- Blockchain integrity checking
- Block synchronization

When a transaction was submitted through the frontend environment, the system successfully:

1. Validated transaction data
2. Generated cryptographic hash values
3. Linked blocks using previous hashes
4. Created blockchain records
5. Propagated blocks across distributed nodes

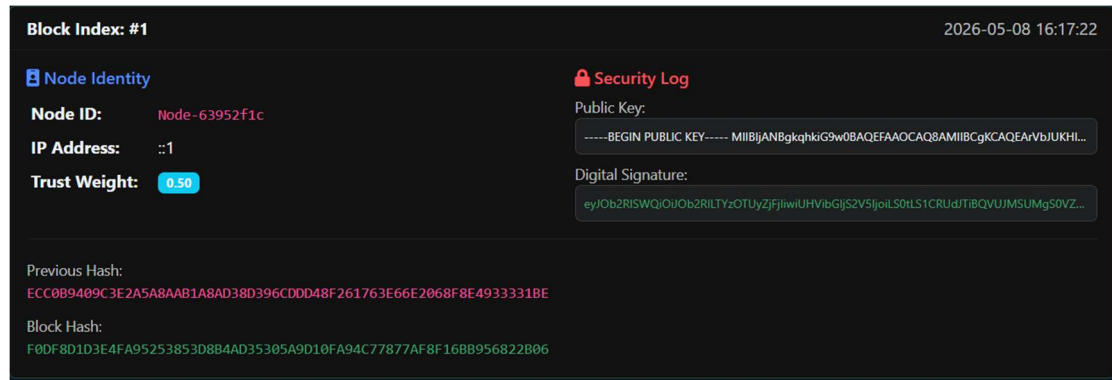
The generated blockchain blocks maintained:

- Immutable transaction records
- Sequential blockchain linkage
- Consistent hash relationships

- Distributed blockchain consistency

The blockchain explorer interface successfully displayed:

- Block information
- Transaction details
- Generated hash values
- Previous block references



**Figure 31 : Blockchain Explorer Displaying Blockchain Transactions and Distributed Records**

The validation results confirmed that the blockchain subsystem successfully prevented unauthorized transaction modification and preserved blockchain integrity across all distributed nodes.

The implemented blockchain validation architecture therefore achieved:

- Data integrity
- Tamper resistance
- Blockchain consistency
- Distributed trust

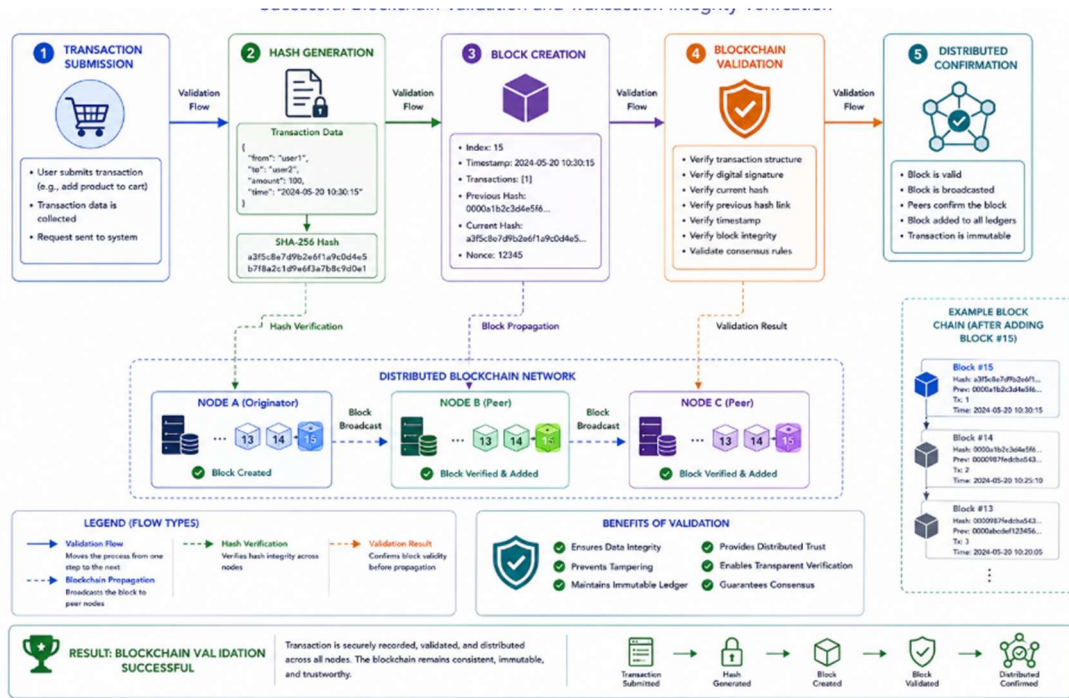


Figure 32 : Blockchain Validation Result Diagram

#### 4-4. Distributed Synchronization Results

The distributed synchronization subsystem was evaluated to verify the consistency and reliability of blockchain replication across all distributed nodes.

The synchronization environment consisted of:

- Node-A
- Node-B
- Node-C

The synchronization results demonstrated that newly generated blocks were successfully broadcasted across all participating nodes.

When a new transaction was processed:

1. A blockchain block was generated
2. The block was broadcasted to peer nodes
3. Neighboring nodes validated the received block
4. Blockchain copies were synchronized
5. Local databases were updated

The synchronization process successfully maintained:

- Consistent blockchain copies
- Distributed transaction history
- Synchronized block records

- Node consistency

The peer-to-peer communication environment additionally verified:

- Successful node communication
- Distributed propagation
- Reliable synchronization
- Multi-node consistency

The synchronization results confirmed that the proposed platform successfully simulated decentralized blockchain network behavior.

The distributed architecture therefore improved:

- Fault tolerance
- Blockchain reliability
- Distributed trust
- Synchronization consistency

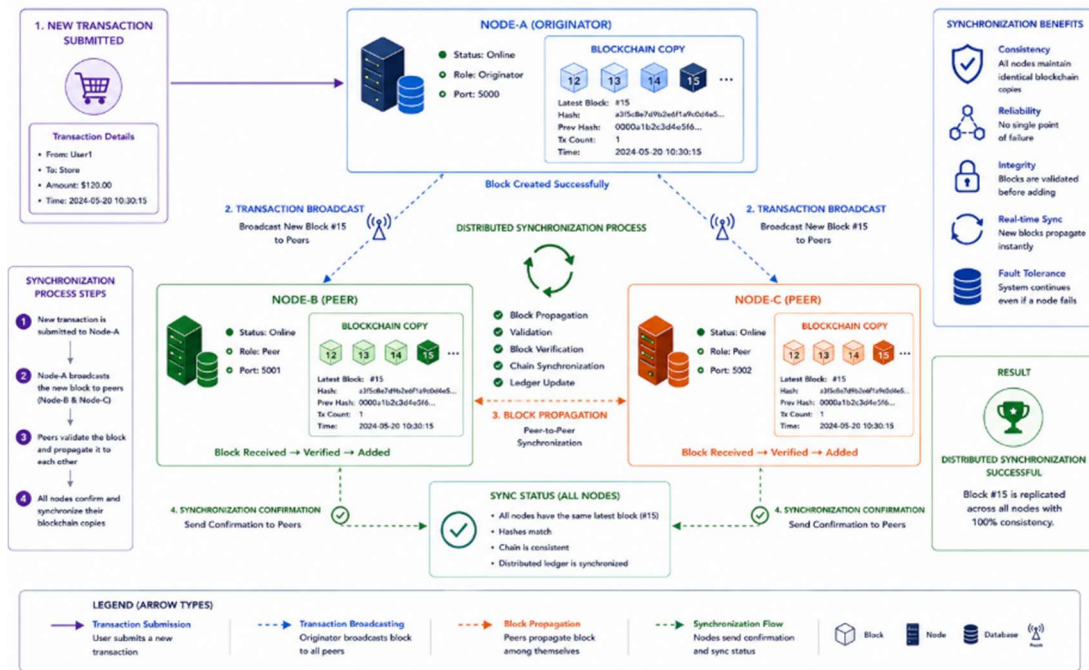


Figure 33 : Distributed Synchronization Result Diagram

## 4-5. REST API Testing Results

The REST API environment was tested using Swagger and HTTP-based request processing in order to verify communication between frontend interfaces, backend services, and distributed blockchain nodes.

The API testing environment successfully handled:

- GET requests
- POST requests
- Transaction submissions
- Blockchain retrieval operations

- Synchronization requests

Swagger successfully provided:

- Interactive endpoint testing
- API documentation
- Request monitoring
- Response validation

The API environment successfully returned:

- JSON responses
- HTTP status codes
- Blockchain data
- Transaction records
- Validation responses

The testing process confirmed:

- Successful controller routing
- Correct API responses
- Stable backend communication
- Secure request handling

The REST API subsystem additionally enabled:

- Node communication
- Blockchain synchronization
- Distributed validation
- Real-time transaction propagation

The API testing results therefore confirmed that the communication layer operated reliably within the distributed blockchain environment.

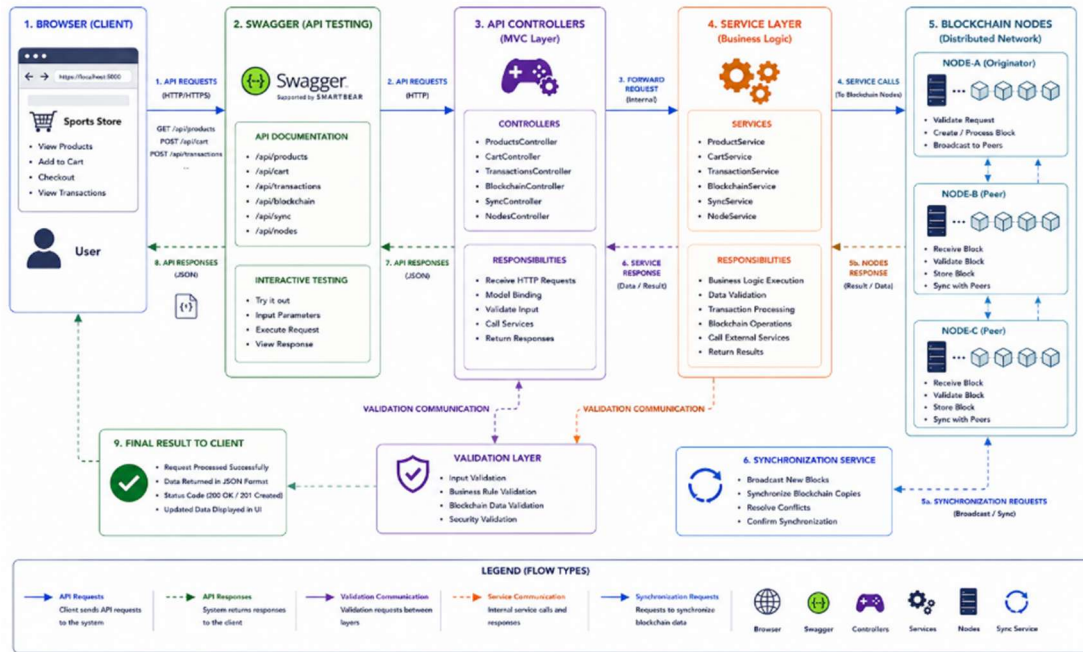


Figure 34 : REST API Result Workflow Diagram

## 4-6. Security and Validation Results

The security subsystem was evaluated to verify the effectiveness of the implemented validation and cryptographic protection mechanisms.

The security testing process confirmed successful:

- Input validation
- Blockchain verification
- Cryptographic hash validation
- Transaction integrity checking
- Audit logging
- Distributed confirmation

The validation workflow successfully rejected:

- Invalid requests
- Incorrect transaction structures
- Unauthorized modifications
- Invalid blockchain references

The implemented cryptographic verification mechanisms successfully ensured:

- Hash consistency
- Blockchain integrity
- Immutable transaction storage
- Tamper resistance

The audit logging subsystem additionally recorded:

- Validation operations
- Transaction processing
- User activities
- Blockchain synchronization events

The security evaluation confirmed that the proposed platform successfully maintained:

- Secure transaction processing
- Blockchain integrity
- Distributed trust
- Traceable operations

The multi-layer validation architecture therefore significantly improved overall system security

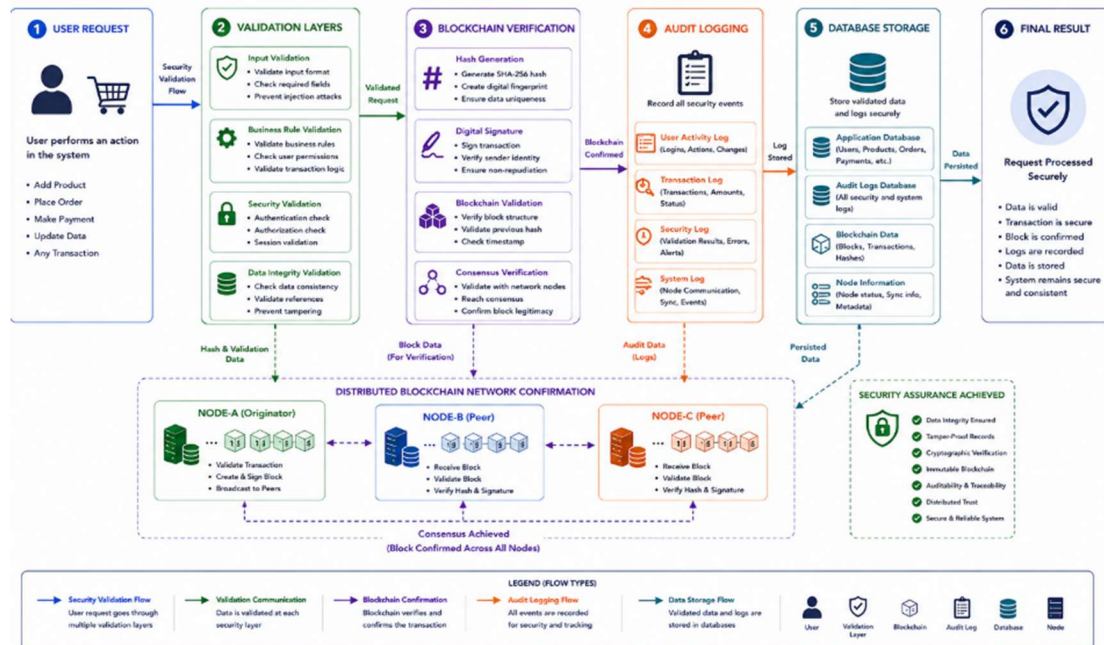


Figure 35 : Security Validation Result Workflow Diagram

## 4-7. Database Persistence Results

The database subsystem was tested to verify reliable storage and retrieval of blockchain and application data.

The system successfully stored:

- Blockchain records
- Product information
- Transaction histories
- Audit logs
- Node metadata
- Synchronization records

Each distributed node maintained:

- Independent database instance
- Local blockchain copy
- Persistent transaction records

The implemented database environment successfully preserved:

- Blockchain consistency
- Transaction persistence
- Distributed synchronization data
- Audit monitoring information

The Entity Framework Core environment successfully handled:

- Entity mapping
- Query execution
- Relationship management
- Database transactions

The database persistence results confirmed that the proposed platform maintained reliable blockchain storage across all distributed nodes.

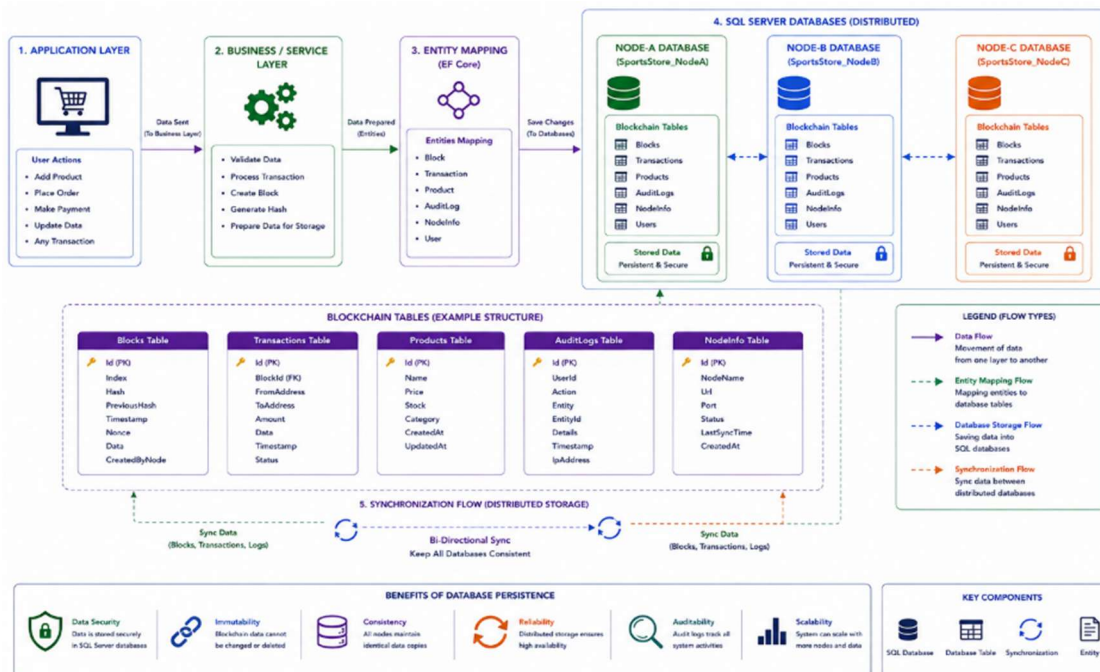


Figure 36 : Database Persistence Result Diagram

#### **4-8. Performance and Stability Results**

The proposed platform demonstrated stable operational performance during distributed execution and blockchain synchronization.

The testing environment confirmed successful:

- Multi-node operation
- Concurrent synchronization
- Stable API communication
- Reliable database connectivity
- Continuous blockchain validation

The distributed architecture successfully maintained:

- Blockchain consistency
- Node synchronization
- Stable communication
- Reliable transaction propagation

The implemented service-oriented architecture additionally improved:

- Modularity
- Maintainability
- Scalability
- Request management

The system remained operational during:

- Multiple transaction submissions
- Distributed synchronization
- Blockchain propagation
- Explorer monitoring

The evaluation therefore confirmed that the proposed platform was capable of operating reliably within a distributed blockchain environment.

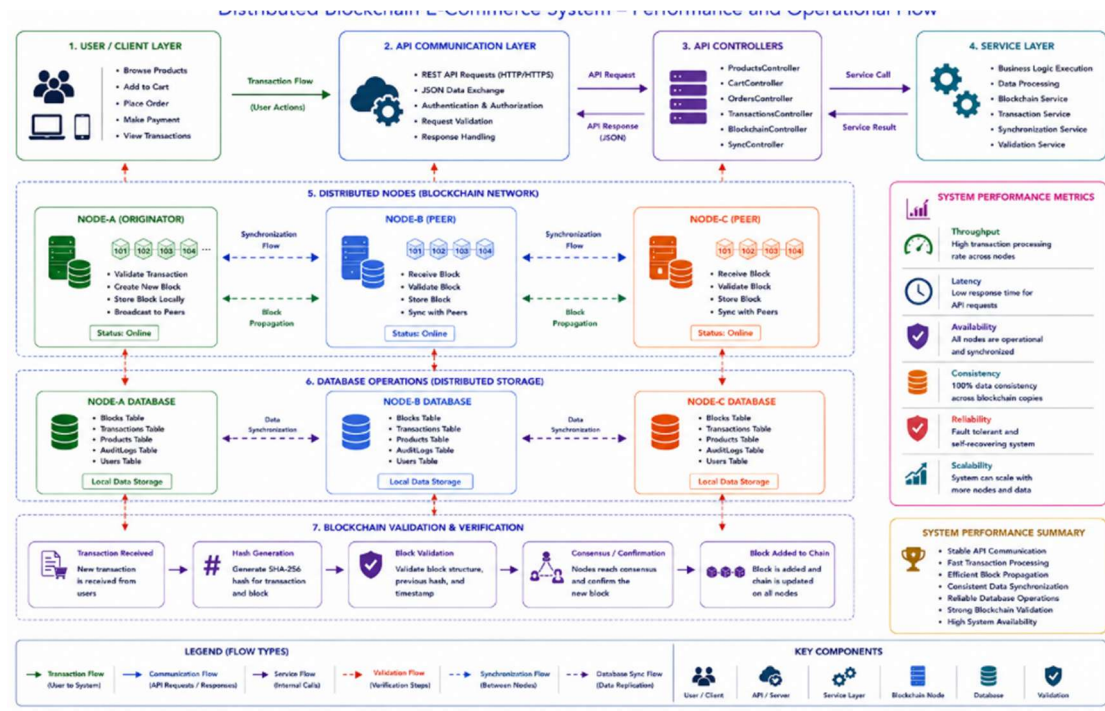


Figure 37 : System Performance Overview Diagram

## 4-9. Comparison with Previous Studies

The proposed platform introduces several improvements compared with traditional centralized systems and previously proposed blockchain-based applications.

The comparison focused on:

- Blockchain validation
- Distributed synchronization
- Security mechanisms
- Multi-node architecture
- Blockchain monitoring
- Audit logging

Table 9 : Functional Comparison of the Proposed System with Traditional and Previous Blockchain-Based Systems

| Feature               | Traditional Systems | Previous Studies | Proposed System |
|-----------------------|---------------------|------------------|-----------------|
| Blockchain Validation | No                  | Partial          | Yes             |
| Distributed Nodes     | No                  | Limited          | Yes             |
| Blockchain Explorer   | No                  | Partial          | Yes             |

|                            |         |         |                  |
|----------------------------|---------|---------|------------------|
| Multi-Node Synchronization | No      | Partial | Yes              |
| Audit Logging              | Limited | Limited | Advanced         |
| MVC Integration            | Basic   | Partial | Full Integration |
| REST API Communication     | Basic   | Partial | Advanced         |
| Distributed Databases      | No      | Limited | Yes              |
| Cryptographic Verification | Limited | Partial | Advanced         |
| Blockchain Replication     | No      | Limited | Yes              |

The comparison results demonstrated that the proposed solution successfully integrated:

- Distributed blockchain validation
- Multi-node synchronization
- Advanced security verification
- Distributed persistence
- Real-time blockchain monitoring

The implemented architecture therefore provided significant improvements over traditional centralized transaction systems.

#### **4-10. Discussion and Analysis**

The obtained results demonstrated that the proposed blockchain-based platform successfully achieved the primary objectives of the project.

The implementation successfully integrated:

- Blockchain technology
- Distributed synchronization
- MVC architecture
- REST API communication
- Database persistence
- Security validation

The blockchain subsystem successfully improved:

- Data integrity
- Transaction immutability
- Distributed trust
- Tamper resistance

The distributed architecture additionally improved:

- Synchronization reliability

- Fault tolerance
- Blockchain consistency
- Scalability

The MVC architecture improved:

- System organization
- Maintainability
- Modularity
- Separation of responsibilities

The evaluation results confirmed that the proposed platform provided a secure and scalable distributed transaction processing environment suitable for blockchain-based e-commerce systems.

The implemented platform therefore achieved the intended security and decentralization objectives of the project.

## **4-11. Conclusion**

This chapter presented the implementation results and evaluation outcomes of the proposed blockchain-based e-commerce security platform. The chapter discussed system execution results, blockchain validation, distributed synchronization, REST API communication, security verification, database persistence, and operational stability.

The obtained results confirmed that the proposed platform successfully implemented distributed blockchain validation, secure transaction processing, blockchain synchronization, cryptographic verification, and decentralized data persistence across multiple blockchain nodes.

The comparison with traditional systems and previous studies demonstrated that the proposed solution introduced several improvements including distributed synchronization, blockchain replication, advanced validation mechanisms, and real-time blockchain monitoring capabilities.

Overall, the implemented platform successfully achieved the project objectives and provided a secure, scalable, and decentralized blockchain-based transaction environment suitable for modern distributed systems.

## **Chapter 6: Conclusion**

## **6-1. Introduction**

This chapter presents the final conclusions obtained from the implementation and evaluation of the proposed blockchain-based e-commerce security platform. It summarizes the major achievements of the project, highlights the technical contributions introduced by the proposed solution, discusses the strengths and limitations of the implemented system, and presents future recommendations for further development and research.

The project aimed to design and implement a secure distributed e-commerce environment capable of improving transaction integrity, decentralized validation, and blockchain synchronization using modern blockchain technologies integrated with web application architecture.

The proposed solution successfully combined blockchain mechanisms with distributed system concepts in order to provide secure transaction processing, blockchain replication, distributed trust management, and tamper-resistant transaction storage.

## **6-2. Project Summary**

The implemented project introduced a distributed blockchain-based e-commerce platform developed using:

- ASP.NET Core
- Entity Framework Core
- SQL Server LocalDB
- MVC Architecture
- RESTful APIs
- Blockchain Services
- Distributed Node Communication

The system successfully integrated:

- Blockchain validation
- Distributed synchronization
- Cryptographic verification
- Blockchain replication
- Audit logging
- Multi-node communication

The proposed platform simulated a decentralized blockchain environment composed of several interconnected nodes operating through peer-to-peer synchronization mechanisms.

Each node maintained:

- Independent blockchain copy
- Local database instance
- Transaction records
- Synchronization services

The project additionally implemented:

- Blockchain Explorer
- Shopping cart system
- REST API communication
- Distributed validation workflows
- Multi-layer security architecture

The implementation demonstrated how blockchain technology can improve transaction security and data integrity within distributed e-commerce systems.

### **6-3. Main Contributions of the Project**

The proposed solution introduced several important technical contributions compared with traditional centralized systems and previous blockchain-based implementations.

The primary contributions of the project include:

#### **6-3.1. Distributed Blockchain Architecture**

The project implemented a distributed multi-node blockchain environment where several independent nodes cooperate to maintain synchronized blockchain copies.

This architecture improved:

- Decentralization
- Fault tolerance
- Blockchain consistency
- Distributed trust

#### **6-3.2. Blockchain Validation Mechanisms**

The platform implemented blockchain validation workflows capable of:

- Hash generation
- Previous hash verification
- Transaction validation
- Block creation
- Blockchain integrity verification

This validation architecture significantly improved transaction immutability and tamper resistance.

### **6-3.3. Distributed Synchronization**

The project successfully simulated peer-to-peer blockchain synchronization between multiple distributed nodes.

The synchronization mechanisms enabled:

- Block propagation
- Blockchain replication
- Distributed confirmation
- Real-time synchronization

### **6-3.4. MVC and Blockchain Integration**

One of the major contributions of the project was the integration between:

- MVC Architecture
- Blockchain services
- REST APIs
- Distributed communication layers

This integration improved:

- Modularity
- Maintainability
- Scalability
- System organization

### **6-3.5. Security and Audit Logging**

The platform implemented several security-oriented mechanisms including:

- Cryptographic verification
- Validation layers
- Audit logging
- Distributed verification
- Blockchain integrity checks

These mechanisms improved:

- Security monitoring
- Traceability
- Data integrity
- Transaction reliability

## **6-4. Strengths of the Proposed System**

The implemented platform demonstrated several important strengths during testing and evaluation.

The major strengths include:

- Secure blockchain validation
- Distributed synchronization
- Decentralized architecture
- Real-time blockchain replication

- Multi-layer validation
- Blockchain explorer monitoring
- Distributed database persistence
- REST API communication
- Modular MVC architecture

The distributed architecture additionally improved:

- Reliability
- Scalability
- Maintainability
- Fault tolerance

The integration of blockchain technology significantly enhanced:

- Transaction immutability
- Data integrity
- Tamper resistance
- Distributed trust

The evaluation results confirmed that the proposed system successfully achieved its primary objectives and provided stable distributed blockchain functionality.

## **6-5. Limitations of the Project**

Despite the successful implementation and testing results, several limitations were identified during the development process.

Despite the successful implementation and testing results, several limitations were identified during the development process.

The main limitations include:

- The project operates within a simulated distributed environment rather than a real public blockchain network.
- The synchronization process was implemented using a simplified distributed communication model.
- The system was tested on a local development environment with limited scalability evaluation.
- Advanced blockchain consensus algorithms such as Proof-of-Work or Proof-of-Stake were not fully implemented.
- Performance testing under extremely high transaction loads was not conducted.
- The blockchain network was limited to a small number of nodes

Additionally, the implemented cryptographic environment focused mainly on educational and research-oriented blockchain simulation rather than enterprise-scale blockchain deployment.

However, these limitations do not reduce the practical and educational value of the project, as the primary objective was to demonstrate secure distributed blockchain concepts within a modern web-based architecture.

## **6-6. Future Work and Recommendations**

Several improvements and future development opportunities were identified during the implementation and evaluation process.

Future enhancements may include:

### **6-6.1. Advanced Consensus Algorithms**

Future versions of the platform may implement:

- Proof-of-Work (PoW)
- Proof-of-Stake (PoS)
- Byzantine Fault Tolerance (BFT)

in order to improve distributed consensus mechanisms.

### **6-6.2. Cloud-Based Deployment**

The system may be deployed using cloud platforms in order to:

- Improve scalability
- Support larger blockchain networks
- Enable internet-based node communication
- Increase fault tolerance

### **6-6.3. Smart Contract Integration**

Future development may include smart contract functionality capable of automating:

- Transaction validation
- Order processing
- Payment verification
- Access control

### **6-6.4. Advanced Cryptographic Security**

Additional security enhancements may include:

- Digital signatures
- Advanced encryption algorithms
- Identity management systems
- Zero-knowledge proof mechanisms

### **6-6.5. Real Blockchain Network Integration**

The proposed platform may be integrated with:

- Ethereum
- Hyperledger
- Polygon
- Binance Smart Chain

in order to support real blockchain deployment environments.

#### **6-6.6. Artificial Intelligence Integration**

Future systems may integrate artificial intelligence techniques for:

- Fraud detection
- Transaction analysis
- Threat prediction
- Behavioral monitoring

### **6-7. Personal Perspective and Research Insights**

Throughout the development and implementation process, the project demonstrated the significant potential of blockchain technology in improving transaction security and distributed trust within modern web applications.

The project additionally highlighted the importance of combining:

- Distributed systems
- Blockchain technology
- Secure APIs
- MVC architecture
- Database synchronization

The implementation experience also revealed several practical challenges related to:

- Distributed synchronization
- Blockchain consistency
- Multi-node communication
- Validation coordination

These challenges provided valuable technical insights regarding the complexity of distributed blockchain systems and the importance of synchronization reliability.

The project therefore contributed not only to technical implementation experience but also to a deeper understanding of decentralized system architecture and blockchain-based security environments.

### **6-8. Final Conclusion**

In conclusion, the proposed blockchain-based e-commerce security platform successfully demonstrated the integration of blockchain technology with distributed web application architecture in order to provide secure transaction processing and decentralized validation.

The project successfully implemented:

- Distributed blockchain synchronization
- Blockchain validation
- Multi-node communication
- REST API integration
- Cryptographic verification
- Distributed database persistence

The obtained results confirmed that the proposed solution improved:

- Data integrity
- Transaction immutability
- Distributed trust
- System reliability
- Blockchain transparency

The integration of MVC architecture, distributed blockchain services, REST APIs, and database synchronization mechanisms created a scalable and modular blockchain-based platform suitable for modern distributed transaction systems.

Overall, the project achieved its intended objectives and provided a successful practical implementation of blockchain concepts within a secure distributed e-commerce environment.

# References

- [1] M. Zhaofeng, M. Jialin, W. Jihui, and S. Zhiguang, “Blockchain-Based Decentralized Authentication Modeling Scheme in Edge and IoT Environment,” *IEEE Internet of Things Journal*, vol. 8, no. 4, pp. 2116–2123, 2021.
- [2] A. Moinet, B. Darties, and J.-L. Baril, “Blockchain based trust & authentication for decentralized sensor networks,” *arXiv preprint arXiv:1706.01730*, pp. 1–6, 2017.
- [3] N. Shi, L. Tan, W. Li, X. Qi, and K. Yu, “A blockchain-empowered AAA scheme in the large-scale HetNet,” *Digital Communications and Networks*, vol. 7, no. 3, pp. 308–316, 2021.
- [4] M. T. Al Ahmed, F. Hashim, S. Jahari Hashim, and A. Abdullah, “Hierarchical blockchain structure for node authentication in IoT networks,” *Egyptian Informatics Journal*, vol. 23, no. 2, pp. 345–361, 2022.
- [5] S. Hong, “P2P networking based internet of things (IoT) sensor node authentication by Blockchain,” *Peer-to-Peer Networking and Applications*, vol. 13, no. 2, pp. 579–589, 2020.
- [6] Ali, S., et al. (2019). Trust-Chain: A Blockchain-Based Trust Management Framework for the Internet of Things. *IEEE Access*, 7, 10092-10103.
- [7] Khanna, A., et al. (2020). Dynamic Weight-Based Consensus Algorithm for Blockchain-Enabled Systems. *Advances in Computing and Network Communications*, 235-248. Springer, Singapore.
- [8] Tripathi, G., Ahad, M. A., & Casalino, G. (2023). A comprehensive review of blockchain technology: Underlying principles and historical background with future challenges. *Decision Analytics Journal*, 9, 100344.
- [9] Dehalwar, V., Kolhe, M. L., Deoli, S., & Jhariya, M. K. (2022). Blockchain-based trust management and authentication of devices in smart grid. *Cleaner Engineering and Technology*, 8, 100481.
- [10] Nakamoto, S. (2008) 'Bitcoin: A Peer-to-Peer Electronic Cash System'. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>.
- [11] Moosavi, N., Taherdoost, H., Mohamed, N., Madanchian, M., Farhaoui, Y. & Khan, I. U. (2024) 'Blockchain Technology, Structure, and Applications: A Survey', *Procedia Computer Science*, 237, pp. 645-658.
- [12] Mourtzis, D., Angelopoulos, J. & Panopoulos, N. (2023) 'Blockchain Integration in the Era of Industrial Metaverse', *Applied Sciences*, 13(3), p. 1353.
- [13] John, K., Kogan, L. & Saleh, F. (2023) 'Smart Contracts and Decentralized Finance', *Annual Review of Financial Economics*, 15, pp. 45-67.
- [14] Szabo, N. (1996) 'Smart Contracts: Building Blocks for Digital Markets'. [Online].

Available: [http://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart\\_contracts\\_2.html](http://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart_contracts_2.html)

[15] John William, A. D., Rajendran, S., Pranam, P., Berry, Y., Sreedharan, A., Gul, J. & Paul, A. (2022) 'Blockchain Technologies: Smart Contracts for Consumer Electronics Data Sharing and Secure Payment', *Electronics*, 12(1), p. 208.

[16] Li, X., Jiang, P., Chen, T., Luo, X., & Wen, Q. (2020). A survey on the security of blockchain systems. *Future Generation Computer Systems*, 107, 841-853.

[17] Antonopoulos, A. M. (2017). *Mastering Bitcoin: Programming the open blockchain* (2nd ed.). O'Reilly Media.

[18] **Atzei, N., Bartoletti, M., & Cimoli, T. (2017).** A survey of attacks on Ethereum smart contracts (SoK). In *Principles of Security and Trust* (pp. 164-186). Springer.

[19] **Troelsen, A., & Japikse, P. (2022).** *Pro C# 10 with .NET 6: Foundational Principles and Practices in Industrial Language Design*. Apress.

[20] **Freeman, A. (2022).** *Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages* (9th ed.). Apress.